

Multikameravisualiseringsverktyg i realtid

- EagleEye

Real-time multi-camera visualization tool
- EagleEye

Daniel Alchasov
Christopher Claesson
Tilde Lindblad
Björn Lindholm
Kacper Orzel
Oskar Rask
Kevin Rintanen Österblad
Alfred Sjöqvist
Isak Skott

Handledare : August Ernstsson
Examinator : Lena Buffoni

Upphovsrätt

Detta dokument hålls tillgängligt på Internet - eller dess framtida ersättare - under 25 år från publiceringsdatum under förutsättning att inga extraordinära omständigheter uppstår.

Tillgång till dokumentet innebär tillstånd för var och en att läsa, ladda ner, skriva ut enstaka kopior för enskilt bruk och att använda det oförändrat för ickekommersiell forskning och för undervisning. Överföring av upphovsrätten vid en senare tidpunkt kan inte upphäva detta tillstånd. All annan användning av dokumentet kräver upphovsmannens medgivande. För att garantera äktheten, säkerheten och tillgängligheten finns lösningar av teknisk och administrativ art.

Upphovsmannens ideella rätt innefattar rätt att bli nämnd som upphovsman i den omfattning som god sed kräver vid användning av dokumentet på ovan beskrivna sätt samt skydd mot att dokumentet ändras eller presenteras i sådan form eller i sådant sammanhang som är kränkande för upphovsmannens litterära eller konstnärliga anseende eller egenart.

För ytterligare information om Linköping University Electronic Press se förlagets hemsida <http://www.ep.liu.se/>.

Copyright

The publishers will keep this document online on the Internet - or its possible replacement - for a period of 25 years starting from the date of publication barring exceptional circumstances.

The online availability of the document implies permanent permission for anyone to read, to download, or to print out single copies for his/hers own use and to use it unchanged for non-commercial research and educational purpose. Subsequent transfers of copyright cannot revoke this permission. All other uses of the document are conditional upon the consent of the copyright owner. The publisher has taken technical and administrative measures to assure authenticity, security and accessibility.

According to intellectual property law the author has the right to be mentioned when his/her work is accessed as described above and to be protected against infringement.

For additional information about the Linköping University Electronic Press and its procedures for publication and for assurance of document integrity, please refer to its www home page: <http://www.ep.liu.se/>.

Daniel Alchasov

Christopher Claesson

Tilde Lindblad

Björn Lindholm

© Kacper Orzel

Oskar Rask

Kevin Rintanen Österblad

Alfred Sjöqvist

Isak Skott

Abstract

This bachelor thesis presents the EagleEye project, a real-time visualization tool designed for multi-camera systems, developed for Axis Communications at Linköping University. The aim of the project was to create a system demonstrating the analytical capabilities of Axis cameras, specifically leveraging their new features for geo-coordinate metadata. Through low-latency video streaming, movement analysis via heatmaps, and alarm functionalities, EagleEye effectively showcases the commercial potential of these cameras.

The project was executed using agile methodologies, combining iterative sprints with Kanban, pair programming, and workshops to ensure efficiency and quality. The EagleEye system comprises a Python-based backend and a TypeScript/React frontend, integrating third-party tools like Mosquitto for MQTT communication and RTSPtoWebRTC for converting camera RTSP streams to WebRTC, achieving latency below one second. An initial system anatomy laid the technical foundation, while regular meetings with Axis communications representatives ensured alignment with their requirements.

EagleEye facilitates efficient product demonstrations of Axis communications camera systems through metadata visualization, highlighting analytical capabilities and commercial applications. Project experiences underline the importance of flexible planning and clear role distribution.

Sammanfattning

Detta kandidatarbete presenterar projektet EagleEye, ett realtidsvisualiseringsverktyg för multi-kamerasystem, utformat för Axis Communications och framtaget vid Linköpings universitet. Projektets syfte var att skapa ett system som demonstrerar den analytiska kapaciteten av Axis-kameror, särskilt deras nya funktioner för metadata med geokoordinater. Genom låglatens-videoströmning, rörelseanalys via heatmaps och larmfunktionalitet visar EagleEye kamerornas potential i kommersiellt syfte.

Projektets utveckling genomfördes med en agil arbetsmetodik genom iterativa sprintar i kombination med Kanban, parprogrammering och workshops som säkerställde effektivitet och kvalitet. Systemet är uppdelat i en backend skriven i Python och en frontend i TypeScript/React med stöd av tredjepartsverktyg som Mosquitto för MQTT-kommunikation och RTSPtoWebRTC för konvertering av kamerornas RTSP-strömmar till WebRTC, vilket möjliggör en latens under en sekund. En initial systemanatomy lade grunden för projektets tekniska struktur, medan regelbundna möten med kontaktpersoner hos Axis säkerställde att verktyget uppfyllde Axis krav på slutprodukten.

EagleEye underlättar effektiva produktdemonstrationer av Axis kamerasystem genom visualisering av metadata, vilket visar kamerornas analytiska kapabilitet och kommersiella tillämpningar. Erfarenheter från projektet betonar vikten av flexibel planering och tydlig rollfördelning. För framtida utveckling föreslås förbättrad systemintegration och en mer användarvänlig design för att ytterligare höja verktygets värde vid produktdemonstrationer.

Författarens tack

Vi vill börja med att rikta ett stort tack till **Axis Communications** för möjligheten att genomföra ett intressant och lärorikt projekt. Projektmedlemmarna har blivit väl bemötta och är tacksamma för det stöd och den hjälp vi har fått under arbetets gång.

Ett särskilt tack riktas till **Sofia Raismaa**, **Rasmus Rynell** och **Kevin Bärudde** för ett mycket professionellt och engagerat bemötande. Projektgruppen är mycket tacksam för alla de tips och idéer ni har delat med er av under projektets gång.

Vi vill även rikta ett varmt tack till vår handledare **August Ernstsson** för stöd under hela projektperioden. Den konstruktiva respons vi har fått på kursens olika moment har varit mycket värdefull, och den kvalitet som rapporten uppnått är i stor utsträckning tack vare detta stöd.

Innehåll

Abstract	iii
Sammanfattning	iv
Författarens tack	v
Innehåll	vi
Figurer	viii
Tabeller	ix
Terms and Definitions	x
1 Introduktion	1
1.1 Motivering	1
1.2 Syfte	2
1.3 Kunden och kundens syfte	2
1.4 Frågeställningar	2
1.5 Avgränsningar och begränsningar	3
1.6 Kontext	3
2 Bakgrund	7
2.1 Kundens bakgrund	7
2.2 Uppdrag	7
2.3 Projektets kontext	8
2.4 Gruppmedlemmarnas tidigare erfarenheter	8
3 Teori	9
3.1 Kommunikationsprotokoll	9
3.2 Webbutveckling	10
3.3 Utvecklingsmetodik och verktyg	10
3.4 Övriga teknologier	11
4 Metod	12
4.1 Utvecklingsmetodik	12
4.2 Arbetsprocess	14
4.3 Metod för att besvara frågeställningen	15
5 Resultat	16
5.1 Implementationens värde	16
5.2 Projektets erfarenheter	21
5.3 Systemanatomin påverkan	23
5.4 Förbättring av sprintplanering	25

6	Diskussion	27
6.1	Resultat	27
6.2	Metod	30
6.3	Arbetet i ett vidare sammanhang	34
7	Slutsats	41
7.1	Hur kan EagleEye-systemet implementeras för att skapa värde för kunden, Axis Communications?	41
7.2	Vilka erfarenheter, som kan vara intressanta för framtida projekt, kan dokumenteras från EagleEye-projektet?	41
7.3	Vilket stöd kan fås genom att skapa och följa upp en systemanatomi?	42
7.4	Hur kan sprintplanering optimeras för att balansera utveckling, testning och dokumentation inom varje iteration?	42
	Litteratur	43

Figurer

5.1	Systemtopologi	17
5.2	LiveView: En realtidsöverblick av objekt och deras positioner	18
5.3	HeatMap: En visualisering av historisk data från objekt	19
5.4	Alarms: Ett verktyg för att sätta upp virtuella larmområden i verktyget.	19
5.5	Gruppens feedback om mål och prioriteringar inom agil arbetsmetodik	21
5.6	Gruppens feedback om arbetsbelastningen	22
5.7	Gruppens feedback om mobbprogrammering	22
5.8	Gruppens feedback om workshops	23
5.9	Systemanatomin för projektet EagleEye	24
6.1	SusAD diagram för den sociala dimensionen	35
6.2	SusAD diagram för den ekonomisk dimensionen	36
6.3	SusAD diagram för den tekniska dimensionen	37
6.4	SusAD diagram för den miljömässiga dimensionen	38
6.5	SusAD diagram för den individuell dimensionen	39

Tabeller

1.1	Projektets avgränsningar och begränsningar	3
1.2	Projektets alla dokument	4
1.3	Projektets medlemmar och roller	4
4.1	Testmatris för krav och tester	13

Termer och definitioner

I det här kapitlet förklaras centrala tekniska termer och begrepp som används i rapporten. Syftet är att ge läsaren en gemensam förståelse för den terminologi som förekommer i beskrivningar av systemets uppbyggnad, funktion och implementation. Definitionerna är anpassade till kontexten för detta projekt, EagleEye, och inkluderar såväl allmänna tekniska termer som mer specifika verktyg och protokoll som använts.

Scen-metadata	Data som genereras av Axis Q1656-kameran i EagleEye, såsom rörelse- och objektinformation i varje videobild, används för att skapa heatmaps och andra visualiseringar.
Axis Q1656	En högupplöst kamera från Axis Communications som används i EagleEye för att fånga videoströmmar och scen-metadata, såsom rörelse- och objektinformation, för realtidsvisualisering.
MQTT	Ett lättviktigt meddelandeprotokoll som används i EagleEye för att publicera och prenumerera på realtidsdata, såsom scen-metadata, mellan kameror och backend.
MQTT-broker	En central serverkomponent i MQTT-protokollet som hanterar all meddelandeförmedling mellan klienter.
Python	Programmeringsspråket som används i EagleEye för backend-utveckling, inklusive hantering av videoströmmar, MQTT-kommunikation och databasoperationer.
Backend	Serversidan av EagleEye-systemet, implementerad i Python, som hanterar videoströmsbearbetning, datalagring och kommunikation med MQTT.
Homografi	En matematisk transformation som kartlägger objektpositioner från kamerans bildvy till en kartvy, använd för att förenkla positionering utan exakta geokoordinater i inomhusmiljöer.
Bounding box	En rektangulär ram som omsluter ett identifierat objekt i en videostrom, använd i EagleEye för att spåra objekt baserat på scen-metadata från Axis Q1656-kameror. Mitten av den nedre kanten används vid homografisk transformation för att kartlägga objektpositioner till en kartvy.
JavaScript	Ett skriptspråk som används i EagleEye som grund för TypeScript i frontend-utvecklingen, särskilt för att hantera dynamiska element i React-gränssnittet.
TypeScript	Ett språk som utökar JavaScript med statisk typning, används i EagleEye för att utveckla en robust och skalbar frontend med React.

Frontend	Användargränssnittet i EagleEye, byggt med React och TypeScript, som visar realtidsvisualiseringar av videoströmmar, heatmaps baserade på kameradata och hantering av larmzoner.
Geokoordinat	Geografiska koordinater (latitud och longitud) som används i EagleEye för att ange kamerapositioner och visualisera metadata, men ersattes med relativa koordinater inomhus på grund av opålitliga GPS-signaler.
Git	Versionshanteringssystemet som användes i EagleEye för att spåra kodändringar, hantera funktionsgrenar och underlätta samarbete mellan utvecklare.
Grenar	Separata utvecklingslinjer i Git, kallade funktionsgrenar i EagleEye, som möjliggjorde parallell utveckling av nya funktioner utan att påverka huvudgrenen.
Heatmap	En visuell representation i EagleEye som använder färger för att visa rörelseintensitet baserat på scen-metadata från Axis Q1656-kameror, vilket underlättar analys av rörelsemönster.
IoT	Internet of Things, ett nätverk av uppkopplade enheter som i EagleEye inkluderar Axis Q1656-kameror och servrar som kommunicerar via MQTT för realtidsdataöverföring.
Kanban	En agil metodik som används i EagleEye för att visualisera och hantera arbetsflödet genom en Kanban-tavla, vilket underlättade planering och uppföljning av uppgifter.
Merge	Processen i EagleEye där kodändringar från funktionsgrenar integreras i huvudgrenen i Git, vilket säkerställde att nya funktioner testades innan de lades till i systemet.
Plug and play	En egenskap i EagleEye-systemet som gör att nya kameror automatiskt kan integreras och börja strömma data utan omfattande manuell konfiguration.
React	Ett JavaScript- och TypeScript-ramverk som används i EagleEye för att bygga ett responsivt och interaktivt användargränssnitt för realtidsvisualiseringar.
Realtidsvisualisering	Målet med EagleEye-systemet att visa videoströmmar och metadata, såsom heatmaps, i realtid med minimal fördröjning, uppnått genom WebRTC och effektiv databehandling.
RTSP	Real-Time Streaming Protocol, används i EagleEye för att strömma video från Axis Q1656-kameror, konverteras till WebRTC för att minska latens i visualiseringen.
Sprint	En tidperiod för implementation där specifika uppgifter och mål planerades och genomfördes enligt agil metodik, vilket strukturerade projektets utveckling.
Switch	En nätverksenhet i EagleEye som kopplar samman Axis Q1656-kameror, servrar och andra enheter i det lokala nätverket för att dirigera datapaket.

TCP	Transmission Control Protocol, används i EagleEye som grund för WebSocket-kommunikation mellan frontend och backend för tillförlitlig dataöverföring.
WebSocket	Ett protokoll som används i EagleEye för kontinuerlig, tvåvägskommunikation mellan frontend och backend över en TCP-anslutning, vilket möjliggör realtidsuppdateringar.
HLS	Ett strömningsprotokoll utvecklat av Apple som delar upp video i små segment som skickas över HTTP.



1 Introduktion

Denna rapport redovisar projektet "Multikameravisualiseringsverktyg i realtid", även kallat EagleEye. Målet var att, med hjälp av ett webbgränssnitt som använde sig av RTSP (Real-Time Streaming Protocol) och geokoordinater, skapa ett visualiseringsverktyg. Verktöget skulle innehålla funktioner som en heatmap, realtids-objektspårning och larmgränser på en karta. Projektet syftade till att underlätta övervakning av ett område och, med hjälp av metadata via MQTT, ge en bättre överblick över identifierade objekt. Metadatan innehåller information om de objekt som kamerorna identifierat, såsom geokoordinater och unika attribut. Vidare kommer rapporten att redogöra för arbetsprocesser, resultat samt erfarenheter som gruppen samlat under projektets gång. Projektet genomfördes av nio studenter vid Linköpings universitet. Detta kapitel redogör för projektets motivering och syfte samt presenterar väsentlig information om projektet.

1.1 Motivering

I takt med att användningen av nätverksbaserade övervakningskameror ökar, växer också behovet av intelligenta system som kan bearbeta och presentera den stora mängden data som genereras. Vid övervakning kan det vara svårt att få en helhetsbild av en situation där flera kameror är inblandade.

Axis Communications, som är en ledande aktör inom nätverksbaserad övervakning, har identifierat ett behov av en lösning som kan visualisera rörelser och objekt över flera kameravyer i en sammanhängande översiktsvy. I samband med det identifierade behovet hade Axis Communications även utvecklat nya tekniker som använder sig av geokoordinater.

En lösning för en tydlig översiktsvy kan förbättra kunders möjligheter att analysera och förstå rörelsemönster, vilket kan vara avgörande inom exempelvis säkerhet, trafikövervakning och butiksanalys. Genom att utveckla ett realtidsverktyg för visualisering av flera kameror, kombinerat med tillhörande scen-metadata, kan en informativ översikt skapas. Denna informativa översikt kan inkludera information som rörelsespår genom objektspårning, men också historiska rörelsemönster i form av en heatmap.

EagleEye syftar till att utveckla denna lösning genom att kombinera kameraövervakningsmetoder med en ny teknik som använder geokoordinater. För att strukturera utvecklingen av EagleEye och tydliggöra systemets komponenter har gruppen använt sig av en

systemanatomi som verktyg för kommunikation och planering. Förhoppningen var att experimentera med de nya teknikerna och skapa en bra översiktsvy för övervakning.

Genom att arbeta iterativt och använda agila metoder, såsom sprintar, säkerställde projektgruppen att systemet uppfyllde de krav och förväntningar som ställdes av beställaren. Vidare användes de agila metoderna för att iterativt utveckla, testa och dokumentera systemet.

1.2 Syfte

Syftet med rapporten är att redovisa och dokumentera projektet EagleEye. Rapporten beskriver de erfarenheter som samlats in samt besvarar projektets frågeställningar.

Målet med projektet EagleEye var att utveckla ett realtidsvisualiseringsverktyg som sammanställde data från flera olika kameror och visualiserade det i en användarvänlig vy. Genom att kombinera video och scen-metadata kunde systemet göra följande:

- Visa en översiktsvy där rörelser kan följas i realtid över flera kameravyer.
- Integrera scen-metadata med objektidentifiering och spårning av objekt.
- Utföra en historisk analys av rörelsemönster genom heatmaps.
- Objektigenkänning som möjliggör identifiering och spårning av samma objekt över flera kameravyer, där systemet avgör huruvida det är samma objekt som rör sig i olika videoflöden.
- Möjliggöra visualisering och spårning av objekt baserat på deras geokoordinater.

1.3 Kunden och kundens syfte

Axis Communications är ett svenskt företag grundat 1984 i Lund. Företaget utvecklar och tillhandahåller nätverkslösningar främst genom kameraövervakning. Lösningarna används inom en rad olika sektorer, främst genom att förbättra säkerhet, driftseffektivitet eller affärsintelligens.

Kundens syfte med projektet är främst att undersöka hur deras nya teknologi, med geopositioner av objekt, kan användas. Projektet fungerar därmed som en utvärdering av hur väl geopositioner kan användas i en praktisk implementation. Dessutom har Axis Communications som avsikt att visa upp projektet i säljsyfte.

1.4 Frågeställningar

Rapporten adresserar följande frågeställningar:

1. Hur kan EagleEye-systemet implementeras för att skapa värde för kunden, Axis Communications?
2. Vilka erfarenheter, som kan vara intressanta för framtida projekt, som EagleEye-projektet genererat och som kan användas i framtida projekt?
3. Vilket stöd kan fås genom att skapa och följa upp en systemanatomi?
4. Hur kan sprintplanering optimeras för att balansera utveckling, testning och dokumentation inom varje iteration?

1.5 Avgränsningar och begränsningar

Projektet omfattades av flera begränsningar och avgränsningar, som presenteras i tabell 1.1.

Tabell 1.1: Projektets avgränsningar och begränsningar

ID	Beskrivning
AV.1	Gruppen har skrivit på ett sekretessavtal vilket medför att den tekniska informationen som gruppen kan dela är begränsad.
AV.2	Systemet behöver hantera minst tre kameror, men med en möjlighet att expandera till fler.
AV.3	Varje gruppmedlem är begränsad till 400 timmars arbetstid.
AV.4	Projektet genomförs under kursens gång, en period på cirka fem månader.
AV.5	Projektet använder endast den utrustning och teknik som Axis Communications och Linköpings universitet har tillgång till eller kan införskaffa.

1.5.1 GDPR

Eftersom projektet utvecklats enbart som ett demoverktyg för Axis Communications och deras kameror, och inte behandlar några användaruppgifter, omfattas projektet inte av specifika krav enligt General Data Protection Regulation (GDPR). Under testerna har åtgärder vidtagits för att undvika att filma offentligheten, vilket minimerar eventuella integritetsrisker. Denna inriktning på demonstrationssyfte, utan insamling eller lagring av personuppgifter, innebär att projektet inte behöver adressera krav såsom samtycke, dataminimering eller rättigheter för dataägare. Ansvar för att upprätthålla GDPR övergår till Axis Communications och framtida användare om systemet anpassas för verklig användning med personuppgifter, där GDPR då skulle bli aktuellt.

1.6 Kontext

Projektet genomfördes inom ramen för kandidatkursen TDDD96 - Kandidatprojekt i programvaruutveckling vid Linköpings universitet. Kursen riktar sig till studenter inom civilingenjörsprogrammen för mjukvaruteknik och datateknik. Kursen startade den 24 januari och avslutades den 23 maj 2025.

För att stödja projektets genomförande tilldelades gruppen en handledare, vars roll var att bistå med rådgivning, vägledning och gav möjlighet att diskutera idéer och utmaningar som uppstod under arbetets gång.

Kursen krävde att vissa projektspecifika dokument skrevs. Alla dokument och deras beskrivningar kan ses i tabell 1.2.

Tabell 1.2: Projektets alla dokument

Dokument	Beskrivning
Projektplan	Projektplanen beskrev projektets syfte och mål samt teamets roller. Den inkluderade även arbetsmetodik, aktiviteter och milstolpar. Planen gav en överblick av projektets struktur, identifierar tidiga risker och fastställde arbetssättet.
Kvalitetsrapport	Kvalitetsrapporten definierade kvalitets mål och riktlinjer för att säkerställa att projektet uppfyllde krav på kodstandard, dokumentation, versionshanteing och testning. Dokumentet definierade även en process för kvalitetssäkring och hur kvaliteten skulle testas under arbetets gång.
Kravspecifikation	Kravspecifikationen dokumenterade de funktionella och icke-funktionella krav som ställdes på systemet. Specificerade vad systemet skulle kunna göra samt vilka prestanda- och säkerhetskrav som gällde. Kravspecifikationen omarbetades även till användarhistorier eftersom kunden lade stor vikt vid en agil arbetsprocess.
Systemanatomi	Systemanatomin gav en övergripande beskrivning av systemets struktur och komponenter. Visade hur olika delar av systemet samverkade, inklusive viktiga moduler och datakommunikation.
Testplan	Testplanen definierade strategier för testning av systemet, inklusive testfall, testmiljö, testmetoder och kriterier för godkännande. Säkerställde att systemet fungerade enligt specifikationerna.
Arkitektokument	Arkitekturdokumentet förklarade övergripande arkitekturval som huvudkomponenterna, datamodeller, lagring, infrastruktur och säkerhet.
Statusrapporter, 3 st	Statusrapporterna var en periodisk sammanställning av projektets aktuella status, framsteg, eventuella förbättringar samt vad som skulle göras under kommande period.

1.6.1 Ansvarsområden och roller

För att strukturera arbetet tilldelades varje gruppmedlem en specifik roll, vilka presenteras nedan i tabell 1.3.

Tabell 1.3: Projektets medlemmar och roller

Roll	Namn
Arkitekt	Daniel Alchasov
Driftsättningsansvarig	Christopher Claesson
Teamledare	Tilde Lindblad
Testledare	Björn Lindholm
Analysansvarig	Isak Skott
Konfigurationsansvarig	Kacper Orzel
Kvalitetssamordnare	Oskar Rask
Utvecklingsledare	Kevin Rintanen Österblad
Dokumentansvarig	Alfred Sjöqvist

1.6.1.1 Teamledare

Teamledaren ansvarade framförallt för gruppen och deras välmående. Teamledaren drev även arbetet framåt med hjälp av mål och delmål som sattes upp tillsammans med gruppen. Vid konflikter vände sig gruppen till teamledaren för medling eller så delegerades ärendet vidare till handledaren vid behov. Under möten agerade teamledaren som ordförande och ansvarade därmed för agenda och mötesprotokoll. Om gruppen inte kom överens hade teamledaren sista ordet. Vidare ansvarade teamledaren för att skicka in veckorapporten till handledaren.

1.6.1.2 Analysansvarig

Analysansvarig höll främst kundkontakten och försökte få fram vilka behov kunden hade samt hur produkten skulle lösa dessa. Rollen krävde kontakt framförallt med arkitekten och även testledaren, för att säkerställa att kundens behov implementerades. Analysansvarig hade dessutom ansvar för kravspecifikationen.

1.6.1.3 Utvecklingsledare

Utvecklingsledaren ansvarade för utvecklingen av produkten genom att driva och leda det tekniska arbetet. Val och beslut gällande implementation, utvecklingsmiljö och design ingick huvudsakligen i utvecklingsledarens arbetsområde.

1.6.1.4 Arkitekt

Arkitekten arbetade för att skapa en välfungerande teknisk arkitektur för produkten genom att identifiera vilka komponenter den bestod av samt att besvara övergripande tekniska frågor. Arkitekt dokumentet var arkitektens huvudansvar.

1.6.1.5 Dokumentansvarig

Dokumentansvarig ansvarade för mallar till samtliga dokument och säkerställde att dessa följde en given standard. Personen hade även ansvar för logotyp, leveranser till deadlines samt samarbetade främst med kvalitetssamordnare och konfigurationsansvarig.

1.6.1.6 Testledare

Testledaren ansvarade för bedömningen av systemets status genom validering av dess funktioner via testning. Tillsammans med kvalitetssamordnaren säkerställdes att kvalitetskraven uppfylldes. Testledaren ansvarade även för testplanen.

1.6.1.7 Kvalitetssamordnare

Kvalitetssamordnaren hade ansvar för kvaliteten på projektet. Inom rollen ingick ansvar för att skriva kvalitetsplanen samt att den följdes. Kvalitetsarbetet genomfördes även av gruppen, men kvalitetssamordnaren hade huvudansvaret.

1.6.1.8 Konfigurationsansvarig

Konfigurationsansvarig ansvarade för versionshantering av projektets dokument och kod. För att säkerställa effektiv hantering valde konfigurationsansvarig lämpliga verktyg för detta ändamål. Arbetet skedde i nära samarbete med dokumentansvarig och utvecklingsledaren.

1.6.1.9 Driftsättningsansvarig

Driftsättningsansvarig ansvarade för att det slutliga projektet blev tillgängligt för kunden. För att säkerställa detta ansvarade personen för distribution, koordinering och installation hos kunden. Under projektets gång arbetade driftsättningsansvarig nära analysansvarig för att säkerställa att projektet följde kundens specifikationer och önskemål.



2 Bakgrund

I denna del presenteras en bakgrund till projektet EagleEye. Först introduceras kunden Axis Communications och deras uppdrag till projektgruppen. Vidare kommer en sammanfattning av projektets kontext. Slutligen följer en kort sammanställning av projektgruppens relevanta erfarenheter inför projektet.

2.1 Kundens bakgrund

Kunden till projektet är Axis Communications. Företagets huvudsakliga roll är att utveckla både hårdvara samt mjukvara i form av övervakningskameror, men också innovativa nätverkslösningar. Dessa bidrar med en ökad säkerhet, trygghet, effektivitet och affärsanalys. Axis ingår sedan 2015 i Canon Group som är ett japanskt multinationellt företag.

Axis Communications är ett svenskt företag som grundades i Lund 1984. Företagets lösningar används globalt inom flera områden. Inom transportsektorn används Axis-kameror exempelvis av BMW Group, där kamerorna övervakar tillverkningsprocessen av BMW-bilar. Med kamerornas inbyggda AI-funktioner fångar de små detaljer som loggans placering, dörrhandtag och liknande, vilket automatiskt kan upptäcka brister i produktionen. För stadsövervakning har kameror från Axis Communications implementerats i staden Okazaki i Japan, där de används för att räkna människor och på så sätt bidra till stadens utveckling mot en mer promenadvänlig miljö (walkable city). Axis lösningar används även inom sjukvården, till exempel på Nura Health Screening Center.

2.2 Uppdrag

Projektgruppens uppdrag av kunden är främst att skapa ett verktyg för att visualisera deras nya teknologi som använder geokoordinater av objekt. Under tiden arbetar gruppen också som pilotanvändare och utvärderar hur bra deras teknik fungerar. Dessutom avser, som nämnts i 1.3, Axis Communications att använda och visa upp detta verktyg i säljsyfte.

Syftet är att skapa ett verktyg som kombinerar kamerans videoström med tillhörande metadata över MQTT, där informationen om igenkänningen av olika objekt överförs. Genom att kombinera dessa parametrar med en karta kan objekten positioneras ut, i deras relativa positioner, på en karta. Vidare finns möjligheten att utveckla tekniker där olika kameror kan identifiera samma objekt.

2.3 Projektets kontext

Som tidigare nämnts i avsnitt 1.5 har projektet ingått i den obligatoriska kursen TDDD96, vilket innebär specifika krav på dokumentation och metodik.

Gruppen har fått hjälp av en tilldelad handledare från universitetet som har påverkat projektets struktur, avstämningar samt rapporteringar.

Ett sekretessavtal har skrivits under med Axis Communications som vidare begränsar den tekniska information som får tas upp i rapporten.

2.4 Gruppmedlemmarnas tidigare erfarenheter

Projektgruppen består av medlemmar från Datateknik och Mjukvarateknik programmet på Linköpings universitet. Från tidigare kurser har samtliga medlemmar erfarenhet av att arbeta i projektgrupper. Gruppmedlemmarna har också erfarenhet av att arbeta med databaser och backend i Python samt ett API byggt med Flask. Gruppmedlemmarna har även genomfört en kurs i programvaruutveckling där de teoretiska delarna av hur en arbetsprocess bör se ut i ett projektarbete tagits upp. Därför har gruppen kunskap om användning av agila metoder såsom Scrum, Kanban och sprintar vid programvaruutveckling.



3 Teori

Detta kapitel syftar till att förklara centrala begrepp för att ge läsaren en teoretisk grund för rapporten. De olika teoretiska delarna presenteras utifrån kommunikationsprotokoll, webb-utveckling, utvecklingsmetodik och verktyg samt övriga tekniska begrepp.

3.1 Kommunikationsprotokoll

Denna del presenterar protokoll som möjliggör kommunikation mellan olika delar av systemet.

3.1.1 RTSP

Real-Time Streaming Protocol (RTSP) är ett applikationsprotokoll som används för att styra leveransen av realtidsmedia, såsom video och ljud, över nätverk. Protokollet används ofta i övervakningssystem där låg latens är avgörande för att kunna ge en aktuell bild av vad som sker [1].

3.1.2 MQTT

För att överföra metadata i realtid används protokollet MQTT (Message Queuing Telemetry Transport) som är ett publikations/prenumerations-baserat protokoll som är optimerat för IoT-enheter och låga bandbreddskrav. Det möjliggör snabb kommunikation mellan kameror och servern [2].

3.1.3 WebRTC

WebRTC (Web Real-Time Communication) är en teknik som gör det möjligt att genomföra röst-, video- och datakommunikation direkt mellan webbläsare och enheter – utan att behöva installera extra program eller plugins. Det används ofta i videosamtal, chattappar och andra realtidsapplikationer, och är en öppen standard som stöds av de flesta moderna webbläsare. WebRTC är ett öppen källkod-projekt som bibehålls av Googles WebRTC team. [3].

3.1.4 RTSPtoWebRTC

RTSPtoWebRTC är ett verktyg som använts för att konvertera RTSP-videoström till WebRTC. Detta verktyg är skrivet i GO och har öppen källkod [4].

3.2 Webbutveckling

Detta avsnitt behandlar teknologier som används för att bygga webbgränssnittet.

3.2.1 React

React är ett JavaScript-bibliotek som används till att bygga användargränssnitt, utvecklat av Meta. React använder en komponentbaserad arkitektur där varje del av gränssnittet är en återanvändbar modul. Genom att använda en virtuell DOM optimerar React renderingen, vilket förbättrar prestandan. React används främst för att skapa interaktiva och dynamiska webbapplikationer [5].

3.2.2 TypeScript

TypeScript är ett programmeringsspråk som bygger på JavaScript, och som adderar stöd för statisk typning [6]. Det betyder att det går att definiera datatyper för variabler och funktioner. Fördelen med definierade variabler och funktioner är att fel kan upptäckas tidigt i utvecklingsfasen. TypeScript-kompilatorn omvandlar sedan koden till vanlig JavaScript som kan köras på webbläsaren eller serversidan. Språket gör koden mer robust och lättläsligt vilket passar bra för större projekt.

3.2.3 DOM

DOM är kort för Document Object Model och är ett gränssnitt som representerar strukturen av ett HTML-dokument som ett hierarkiskt träd av DOM-objekt.

Virtuell DOM är en minnesbaserad representation av det riktiga DOM-trädet i webbläsaren. Genom att först ändra den virtuella DOM:en och sedan jämföra med den verkliga DOM:en, kan bibliotek som React effektivt uppdatera endast de delar av sidan som faktiskt förändrats. Detta förbättrar prestanda och förenklar utvecklingen, eftersom webbläsaren slipper hantera DOM-uppdateringar manuellt [7].

3.2.4 Vite

Vite är ett modernt byggverktyg för JavaScript/TypeScript projekt [8].

3.3 Utvecklingsmetodik och verktyg

Här beskrivs arbetsmetoder och verktyg som har använts under projektets gång.

3.3.1 Scrum och sprintar

Scrum är ett verktyg som används inom agil utvecklingsmetodik. Det är till för att utveckla och leverera programvara på ett iterativt och flexibelt sätt [9]. Scrum bygger på kortare utvecklingscykler som kallas för sprintar, som vanligtvis är mellan en och fyra veckor långa.

3.3.2 Jira

Jira är ett mjukvaruprogram för projektplanering och förvaltning. Programmet tillåter användare att planera, spåra och förvalta agila mjukvaruprojekt [10].

3.3.3 Figma

Figma är ett molnbaserat designverktyg som används för att skapa prototyper till olika mjukvaruprodukter. Verktöget kan köras i webbläsaren och stödjer realtidssamarbete och tillåter användaren att bygga interaktiva designers [11].

3.3.4 Git-pipelines

Git-pipelines, ofta kallade CI/CD-pipelines är en automatiserad process för att bygga, testa och distribuera kodändringar i ett Git-baserat arbetsflöde. Git-pipelines bevisat sig effektivisera agil utveckling och möjliggör en ökad produktivitet i programmeringsprojekt [12].

3.4 Övriga teknologier

Här samlas diverse teknologier och tekniska koncept som används i projektet.

3.4.1 Go

Go, även känt som Golang, är ett modernt, öppet-källkodprogrammeringsspråk utvecklat av Google sedan 2009. Det designades för att vara enkelt, effektivt och skalbart, särskilt för systemprogrammering, molnbaserade applikationer och distribuerade system [13].

3.4.2 Python

Python är ett högnivåspråk med dynamisk typning, känt för sin läsbarhet och relativa enkelhet [14]. Python används inom många olika områden såsom webbutveckling, dataanalys, maskininlärning, automation och systemutveckling. Det har ett stort sortiment av bibliotek och ramverk, vilket gör språket populärt för både nybörjare och erfarna programmerare.

3.4.3 Geokoordinater

En geokoordinat är en geografisk position på jorden som baseras på latitud och longitud [15]. I ett sammankopplat övervakningssystem har varje kamera en bestämd hårdkodad position, medan positionen av rörliga objekt kan beräknas utifrån kamerans geokoordinater.

3.4.4 TailWind CSS

Tailwind CSS är ett "utility-first-ramverk" som förenklar skapandet av responsiv design genom att låta utvecklaren applicera stilklasser direkt i HTML, vilket eliminerar behovet av externa CSS-filer [16].



4 Metod

I detta kapitel redogörs för de metoder, arbetssätt och tekniker som använts under projektets gång. Syftet är att ge en tydlig bild av hur utvecklingsarbetet har strukturerats och genomförts, samt vilka verktyg som använts för att säkerställa kvalitet och effektivitet. Avsnittet omfattar utvecklingsmetodik, arbetsprocesser, val av tekniker och verktyg, teststrategier samt hur teamet arbetat med erfarenhetssamling och förbättring av arbetssätt.

4.1 Utvecklingsmetodik

I detta avsnitt beskrivs den övergripande utvecklingsmetodik som används i projektet, med fokus på en agil arbetsmetod. Avsnittet behandlar val av metod, kommunikation, avstämningar, integration, testning samt parprogrammering. Dessa områden utgör tillsammans grunden som säkerställer en effektiv, hållbar och kvalitetssäkrad utvecklingsprocess.

4.1.1 Inledning

I detta projekt har gruppen valt att arbeta med en agil utvecklingsmetod, där arbetet organiserades i en till två veckor långa sprintar. Perioden för sprintarna bestämdes gemensamt av gruppen baserat på den totala planerade utvecklingstiden för projektet. Vid slutet av varje sprint genomfördes en sammanställning och utvärdering för att analysera arbetsprocessen och produktens dåvarande status. Under dessa utvärderingar bedömdes det om arbetsbelastningen behövde justeras för att säkerställa en balans mellan produktivitet och hållbar arbetsbelastning.

4.1.2 Val av metod

Val av en agil metodik grundades på dess förmåga att säkerställa kontinuerlig leverans av en fungerande produkt. Genom att arbeta i sprintar och sträva efter att ha en stabil version av systemet vid varje sprintavslut, minskas risken att kritiska fel ackumuleras. Många kritiska fel kan leda till en obrukbar produkt. Vidare minimeras risken för omfattande fel genom att arbeta i funktionsgrenar, där varje funktionalitet utvecklas och testas separat innan den integreras i huvudkodbasen. Vid slutet av varje sprint genomfördes tester av hela systemet, vilket ytterligare reducerade risken för allvarliga fel som kan påverka systemets stabilitet.

4.1.3 Kommunikation och avstämningar

Kommunikation och samarbete är centrala aspekter av den valda utvecklingsmetoden. Större avstämningsmöten hölls vid varje sprintavslut för att säkerställa att alla gruppmedlemmar har en gemensam förståelse för projektets status och riktning. Målet med dessa möten var att undvika redundans i arbetet, exempelvis genom att identifiera potentiella överlappningar mellan olika utvecklingsgrupper och funktioner.

4.1.4 Integration

En av de främsta fördelarna med agil utveckling är möjligheten till kontinuerlig integration. Genom att integrera och testa kod regelbundet, främst vid sprintavslut, minimeras risken för omfattande merge-konflikter. Mindre merge-konflikter sparar tid och förbättrar kodens kvalitet. Vidare gjorde detta att utvecklarna kunde fokusera mer på ny funktionalitet istället för att felsöka och hantera konflikter i kodbasen.

4.1.5 Testning

Det implementerades automatiska tester med GitLab-pipelines, vilket säkerställde att all kod som pushas genomgick tester innan den lades till i huvudgrenen. Vidare skedde även manuella tester för vissa delar av systemet. Om testerna misslyckades avvisades ändringarna, vilket förhindrade att icke-fungerande eller otestad kod infördes i systemet.

För att säkerställa att systemet gör och presterar som önskat gjordes testningen enligt testmatrisen som kan ses i tabell 4.1.

Tabell 4.1: Testmatris för krav och tester

Kravbeskrivning	Testtyp	Testfall
Visualisering av RTSP-strömmar i gränssnittet	Funktionellt test	Kontrollerade att videoströmmarna från varje kamera visades i användargränssnittet vid körning av systemet.
Metadatahantering via MQTT	Funktionellt test	Kontrollerade att metadata skickades med korrekt JSON-format till användargränssnittet genom enhetstester.
Översiktsvy med objektplacering på karta	Funktionellt test	Kontrollerade att objekt placerades korrekt utifrån geokoordinater på kartan med testdata och visuell kontroll.
Skapandet av värmekarta från rörelsedata	Funktionellt test	Skapade testdata och verifierade att värmekartan visade rätt rörelsemönster.
Objektspårning	Funktionellt test	Verifierade att systemet korrekt spårade objekten över kamerorna genom manuell testning av systemet.
Realtidskrav: < 1 sek latens	Prestandatest	Mätte latens från visuell information till visning i användargränssnittet.
Plug-and-Play-installation	Kodgranskning	Säkerställde att alla kamerainställningar konfigurerades med en konfigurationsfil.
Körbart på Linux	Funktionellt test	Kontrollerade att systemet var körbart på en Linux-maskin och verifierade funktionalitet.
Prestanda med minst 3 kameror	Belastningstest	Testade att 3 videoströmmar hanterades parallellt.
Modularitet av kodbasen	Kodgranskning	Lät kunden utvärdera kodbasen för framtida användning.

Genom beskrivningarna i figur 4.1 säkerställdes det att varje krav testas och uppfylls.

4.1.6 Arbetssätt

Utvecklingen har skett genom parprogrammering samt mobbprogrammering, där två eller fler utvecklare arbetade tillsammans på en funktionsgren under sprintens gång. Vid sprintens slut integrerades samtliga funktioner och testades i en särskild utvecklingsgren innan de infördes i huvudgrenen. Detta tillvägagångssätt säkerställde att endast stabil och testad kod nådde huvudkodbasen, vilket bidrog till en mer robust och tillförlitlig produkt [17].

4.2 Arbetsprocess

I denna del redogörs information om hur gruppen lade upp sprintarna inom projektet, hur strukturen såg ut samt hur mötena planerades.

4.2.1 Arbetssätt

I detta projekt arbetade teamet i sprintar på en till två veckor. Vid slutet av varje sprint sammanställdes den kod som har utvecklats, integrerats och testats innan den infördes i huvudgrenen. Endast kod som har verifierats som fungerande, genom våra automatiserade tester, tilläts att integreras i huvudkodbasen. För att säkerställa kodkvalitet genomfördes även kodgranskning vid sprintens slut. Eftersom gruppen använde sig av parprogrammering roterade paren. Vilket innebär att ingen granskar sin egen kod. Detta säkerställde att minst fyra personer granskat varje kodavsnitt innan det godkänkts.

4.2.2 Sprintplanering

Varje måndag hölls gruppmöten där det sammanställdes vad som hade utförts under den senaste sprinten och det fortsatta arbetet planerades. Måndagsmötena bidrog till en tydlig kommunikation inom teamet, vilket minskade risken för redundans och säkerställde att alla hade en klar bild av sina arbetsuppgifter. Dessutom gavs gruppmedlemmarna möjlighet att be om stöd vid eventuella problem.

4.2.3 Dokumentation av arbetsuppgifter

Samtliga uppgifter och arbetsmoment registrerades i gruppens Kanban-system, Jira, vilket gav en tydlig översikt över arbetets utveckling och vilka arbetsuppgifter som hade slutförts. Denna transparens underlättade samordning mellan paren och säkerställde en effektiv arbetsprocess.

4.2.4 Kravinsamling

Insamlingen av krav skedde genom en kombination av traditionella och agila kravspecifikationer. Både funktionella och icke-funktionella krav definierades. Den agila kravspecifikationen gjordes genom användarhistorier, vilket var ett krav från Axis Communications. För att undvika onödiga designval skapades en prototyp av systemet i Figma, vilket sedan presenterades och godkändes av Axis Communications innan implementationen påbörjades.

4.2.5 Systemanatomi

Under projektets gång användes systemanatomi som en övergripande planering som säkerställde att kodstrukturen inte gick för långt ifrån de mål och den planering som var skapad. Genom att under programmeringsarbetet blicka tillbaka på systemanatomin kunde gruppen kontrollera att de höll sig inom projektets ramar och var på rätt väg till en slutprodukt.

4.2.6 Kundmöten

Under projektets gång tillämpades ett iterativt arbetssätt med regelbundna återkopplingsmöten tillsammans med kunden. Inför varje möte skrevs ett protokoll som skulle följas av de representanter som deltog på mötet. En stående punkt i protokollen var att presentera aktuella framsteg samt senaste versionen av prototypen. Detta var för att kunden skulle ha möjlighet att kontinuerligt ge återkoppling angående produkten. Återkopplingen dokumenterades och låg till grund för planering av kommande arbetsmoment. Vilket i sin tur bidrog till produkten ständigt anpassades efter kundens behov.

4.3 Metod för att besvara frågeställningen

I denna del beskrivs de metoder och processer som använts för att besvara projektets frågeställningar.

4.3.1 Hur kan EagleEye-systemet implementeras för att skapa värde för kunden, Axis Communications

För att besvara denna frågeställning byggdes ett fungerande realtidsvisualiseringssystem, EagleEye, i nära samarbete med kunden Axis Communications. Projektgruppen har utvecklat en kravspecifikation och iterativa användarhistorier i nära samarbete med kunden. Kundmöten och demonstrationer av projektstatus har hållits regelbundet där kunden haft möjlighet att ge feedback och direktiv till gruppen om hur maximalt värde kan skapas.

4.3.2 Vilka erfarenheter, som kan vara intressanta för framtida projekt, kan dokumenteras från EagleEye-projektet?

För att besvara frågeställningen har projektgruppen samlat erfarenheter under projektets gång. Detta har inkluderat användning av retrospektiv i slutet av sprintar, kontinuerlig dokumentation i Jira samt analys av gruppens interna feedback i form av enkäter gällande arbetsbelastning, prioriteringar, mobbprogrammering och workshops.

4.3.3 Vilket stöd kan fås genom att skapa och följa upp en systemanatomy?

Systemanatomien togs fram tidigt i projektet av tre gruppmedlemmar genom gemensam brainstorming, med involvering av hela teamet. Arbetet utgick från en mall i det bifogade kursmaterialet [18]. Anatomien visualiserades som ett diagram över funktionella beroenden i systemet. Under projektets gång användes systemanatomien som ett planerings- och kommunikationsverktyg.

4.3.4 Hur kan sprintplanering optimeras för att balansera utveckling, testning och dokumentation inom varje iteration?

Projektet har utvecklats med en agil utvecklingsmetodik och sprintarna har genomförts regelbundet. Under sprintarna hade medlemmarna delats upp i sprintgrupper och till en början var det två personer vardera. Senare tilldelades uppgifter för varje grupp. För planering, uppföljning och hantering av uppgifter användes Jira som projektverktyg. Verktöget möjliggjorde en tydlig överblick över sprintarnas innehåll, ansvarsfördelning och framsteg, samt underlättade kommunikationen inom och mellan grupperna. För att besvara frågeställningen har gruppen jobbat retrospektivt och efter varje sprint har ett möte hållits där alla gruppmedlemmar fått chans att ge feedback om sprinten som varit. Denna feedback har sedan använts för att förbättra nästkommande sprint.



5 Resultat

I det här kapitlet beskrivs systemet som har byggts under projektet och hur arbetsmetoderna utvecklades. Det innehåller också de erfarenheter gruppen har fått under arbetets gång.

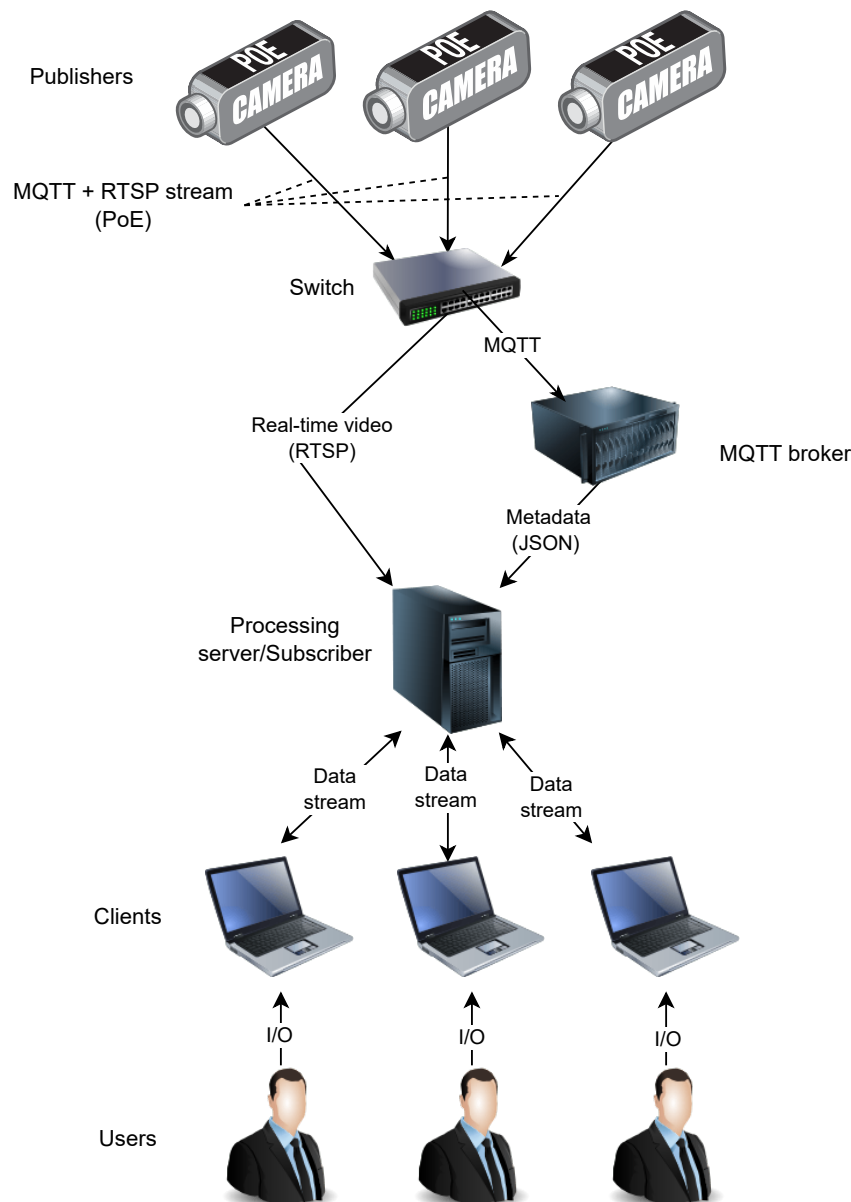
5.1 Implementationens värde

I detta avsnitt presenteras den färdiga implementeringen av EagleEye-systemet och dess värde i förhållande till projektets mål. Fokus ligger på att beskriva hur systemet är uppbyggt, de funktioner som har utvecklats samt vilken nytta det ger för användaren. Även återkoppling från beställaren redovisas för att visa systemets praktiska tillämpning och relevans.

5.1.1 Systemöversikt

I början av projektet togs en systemöversikt fram för att tydliggöra hur EagleEye är strukturerat och hur de olika komponenterna hänger ihop. Figur 5.1 visar hur data flödar från de anslutna Axis Q1656-kamerorna till klienterna via mellanliggande systemkomponenter. Varje kamera fungerar som en sändare och skickar videoströmmar via RTSP samt metadata via MQTT till en switch. Denna switch fördelar vidare videoströmmar till både videolagring och en central bearbetningsserver, medan metadata skickas till en MQTT-broker.

Bearbetningsservern agerar som prenumerant och samlar in både video och metadata, bearbetar informationen och skickar vidare sammanställd data till klienten för visualisering i realtid. Systemet är uppdelat i flera logiska komponenter: kameror, kommunikationsflöde, backendkomponenter och frontend. Denna uppdelning gör det möjligt att skala systemet vid behov och möjliggör parallell utveckling mellan grupper.



Figur 5.1: Systemtopologi

5.1.2 Kommunikationsflöde

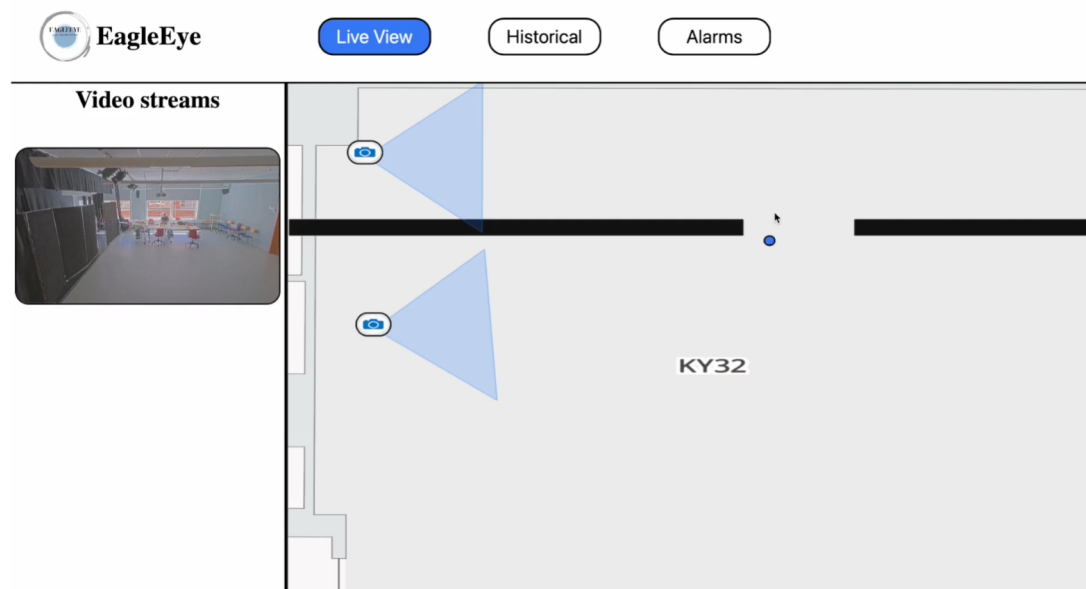
Videoströmmarna transporteras från varje kamera till systemets bearbetningsserver med hjälp av RTSP, ett protokoll för realtidsöverföring av media. Samtidigt skickas metadata via MQTT till en broker som distribuerar dessa meddelanden till prenumeranterna, vilket i detta fall är bearbetningsservern. Genom att separera video och metadatakanaler kan systemet hantera olika krav på latens och bandbredd. RTSP-strömmar kräver hög genomströmning medan MQTT är lätt och optimerat för att snabbt vidarebefordra sensorinformation.

5.1.3 Backendkomponenter

Backendens ansvarar för att ta emot, behandla och skicka vidare information från kamerorna till frontend. När en kamera upptäcker ett objekt skickas metadata som objektets position och typ via MQTT till backend. Samtidigt skickas videoströmmarna separat för att visas i realtid. Backend tar emot metadata, tolkar det, och placerar ut objekt på en karta genom att räkna om positionerna till relativa koordinater. Informationen sammanställs sedan och skickas till frontend, där objektens rörelser visas i realtid. Systemet hanterar även larmzoner, som kan skapas eller tas bort av användaren, och skickar ut varningar när ett objekt går in i ett sådant område.

5.1.4 Frontend

Frontend-applikationen är utvecklad med React och TypeScript och byggs med hjälp av Vite, vilket möjliggör snabb utveckling och en modulbaserad struktur. Gränssnittet är uppdelat i flera vyer som tillsammans ger användaren översikt och kontroll över kamerornas videoströmmar och tillhörande metadata. I LiveView-vyn presenteras användaren för en realtids-karta där objektens positioner uppdateras kontinuerligt. Genom att klicka på ett objekt kan användaren öppna dess tillhörande videoströmmar, som visas direkt i gränssnittet. Objektens positioner uppdateras kontinuerligt, vilket visas i Figur 5.2 nedan.



Figur 5.2: LiveView: En realtidsöverblick av objekt och deras positioner

För att ge en översikt över historiska rörelsemönster har en heatmap-funktion implementerats i komponenten HeatMap. Denna vy renderar ett lager med olika färgintensitet ovanpå planritningen där röda områden markerar hög aktivitet. Vidare indikerar grönt och färger närmare grönt mindre aktivitet. På skärmen kan användaren växla mellan olika tidsintervall som till exempel "Last Hour" eller "24 Hours" för att se hur rörelser har varierat över tid. En helhetsbild av Heatmap-sidan kan ses i Figur 5.3.



Figur 5.3: HeatMap: En visualisering av historisk data från objekt

I Larms-vyn visualiseras larmområden och detekterade objekt som knappar över en bild av planritningen. Denna vy ger en snabb och tydlig indikation på var incidenter inträffat. Larmsidan visas i Figur 5.4 nedan.



Figur 5.4: Alarms: Ett verktyg för att sätta upp virtuella larmområden i verktyget.

All kommunikation mellan frontend och backend sker via WebRTC för videoströmmar och via MQTT för metadata. Gränssnittet är designat för att vara responsivt, lättnavigerat och kunna hantera flera samtidiga anslutningar utan att påverka prestanda som kunden efterfrågar.

5.1.5 Huvudfunktioner och implementation

I implementationen har stor vikt lagts vid att uppfylla de grundläggande funktionella och icke-funktionella kraven enligt kravspecifikationen. Systemet kan hämta och visa videostömmar från flera kameror samtidigt med en fördröjning på under en sekund. Metadata samlas in parallellt, vilket möjliggör att objekt såsom människor eller fordon kan identifieras och placeras ut på kartan. Informationen bearbetas i backenden och skickas vidare till frontend där den visualiseras i ett användarvänligt gränssnitt. Heatmaps genereras genom att analysera tidigare rörelsedata vilket ger användaren möjlighet att identifiera mönster i rörelser över tid. Frontend är byggd i React och kommunicerar via WebRTC för att säkerställa låg latens. Backend är huvudsakligen skriven i Python med stöd för automatiserad databehandling, och är integrerad med ett MQTT-baserat meddelandesystem. All datahantering och kommunikation är utformad för att möjliggöra ett smidigt och realtidsanpassat flöde från kamera till användare.

5.1.6 Mätningar av systemet

Under projektets gång har genomförde gruppen flera typer av mätningar på systemet för att säkerställa dess funktionalitet och prestanda. Fokus har legat på både funktionella tester av olika systemkomponenter och på prestandamätningar, främst i form av latensmätningar.

Två centrala delar av backenden som har testats är komponenterna `Mapmanager` och `Objectmanager`. `Mapmanager` som hanterar den huvudsakliga logiken kopplad till kartvyn, såsom geokoordinater, kamerornas position samt omvandling till relativa-koordinater. `Objectmanager` ansvarar för logiken kring objekt som detekteras av kamerorna. Båda dessa komponenterna har testats med enhetstester, implementerades med hjälp av `unittest`-biblioteket i Python. Testerna identifierade ett fåtal mindre fel som snabbt kunde åtgärdas. Detta gav gruppen förtroende för att fortsätta utvecklingen med vetskapen att dessa delar av systemet fungerade stabilt. Dessutom bidrog testerna till att identifiera specialfall och säkerställa att systemet inte kraschar vid oväntad indata.

Gruppen genomförde även mätningar av latensen i kamerasystemet, vilket är en kritisk faktor för ett övervakningssystem. Målet var att systemet skulle uppvisa en latens på under 1 sekund från det att kameran fångar en händelse till dess att den visas i frontend. Den första implementationen, som använde HLS-protokollet, uppvisade en latens på cirka 10 sekunder. Efter analys och felsökning identifierades protokollet som flaskhalsen. Genom att byta till WebRTC kunde latensen reduceras till cirka 0,5 sekunder enligt manuella tester, vilket gruppen bedömde vara tillräckligt för att uppfylla systemets syfte.

Slutligen testades även larmfunktionerna i backenden. Dessa testades med enkla assertion-tester och egna testfall. Samtliga tester passerade utan fel, vilket överensstämde med gruppens förväntningar då larmmetoden var relativt enkel i sin implementation.

Frontend testades mindre då projektet hade som mål att ha så mycket av logiken som möjligt i backenden. Testerna för frontend har främst varit visuella, där gruppen testade hur den förändras beroende på skärmstorlek. Detta gjordes med ett inbyggt verktyg som låter användaren ändra webbläsarens upplösning. Detta gjordes även genom att starta frontend på skärmar med olika storlek och upplösning. Detta har hjälpt till att förbättra och göra frontend responsivt beroende på skärmstorlek.

Slutligen godkändes alla testfall från tabellen 4.1 förutom det modulära kravet då kund svar inväntas.

5.1.7 Leverans och kundfeedback

Under projektets gång har gruppen haft ett kontinuerligt och nära samarbete med kunden, Axis Communications, för att säkerställa att slutprodukten inte bara möter de tekniska kraven utan även skapar konkret värde i kundens verksamhet. Regelbundna möten har hållits

där hela gruppen deltagit och presenterat det senaste arbetet. Vid dessa tillfällen har kunden fått möjlighet att ge direkt återkoppling på funktionalitet, frontend och prestanda, vilket skapat goda förutsättningar för agilt och behovsdrivet utvecklingsarbete. Samarbetet med kunden har fungerat mycket bra och det har legat som grund till ett framgångsrikt projekt.

5.2 Projektets erfarenheter

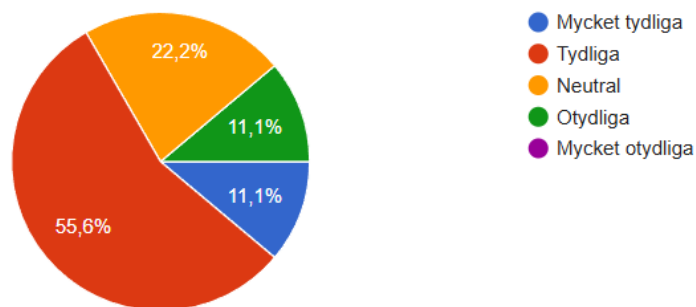
Under arbetet med projektet har gruppen samlat flera värdefulla erfarenheter kring projektstyrning, arbetsmetodik, samarbete inom mjukvaruutveckling. Nedan sammanfattas de viktigaste överförbara erfarenheterna som kan vara till nytta för framtida liknande projekt.

5.2.1 Vikten av flexibel agil arbetsmetodik

En central erfarenhet från projektet är vikten av att välja en agil arbetsmetodik som inte följer en alltför strikt struktur, utan som möjliggör flexibilitet i arbetsfördelningen. Genom att ha ett sprintupplägg där arbetsuppgifter kunde omfördelas vid behov, kunde teamet anpassa sig till förändringar i arbetsbelastning och individers tillgänglighet. Figur 5.5 nedan visar resultat från gruppens svar på en intern enkät om mål och prioriteringar inom agil arbetsmetodik. Resultaten visar att majoriteten av gruppmedlemmarna upplevde att arbetsmetoden gav god anpassningsförmåga och tydliga prioriteringar.

Hur tydliga var sprintens mål och prioriteringar i början av sprinten?

9 svar



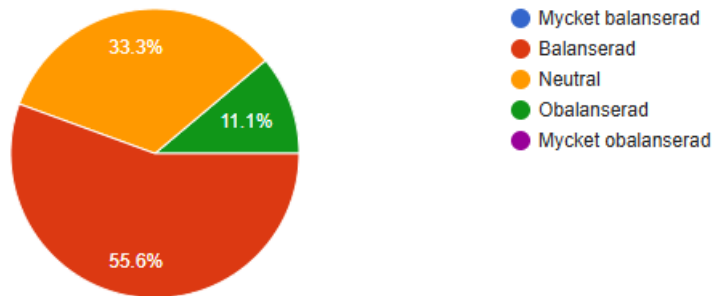
Figur 5.5: Gruppens feedback om mål och prioriteringar inom agil arbetsmetodik

5.2.2 Strukturerad ansvarsfördelning och transparens

Projektet visade också tydligt på vikten av att ha en struktur som synliggör arbetsinsatser från varje individ. Genom att registrera arbetsuppgifter på den gemensamma Kanban-tavlan i Jira för uppföljning, skapades en naturlig form av ansvarstagande där varje medlems bidrag var transparent. Detta främjade en kultur av ansvarstagande och gjorde det enklare att snabbt identifiera och åtgärda eventuella obalanser i arbetsbelastningen. Figur 5.6 nedan visar resultat från gruppens svar på en intern enkät om arbetsbelastningen, där majoriteten av medlemmarna upplevde arbetsbördan som balanserad.

Hur balanserad var din arbetsbelastning under sprinten?

9 responses



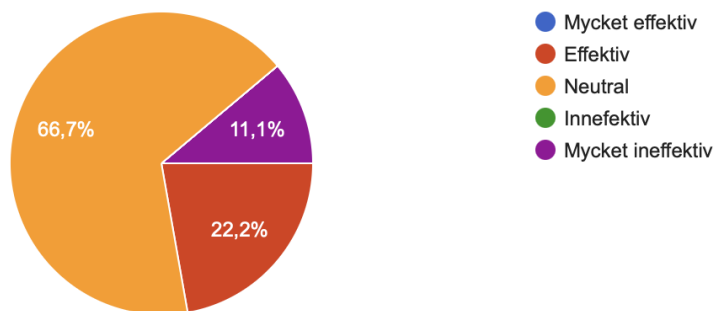
Figur 5.6: Gruppens feedback om arbetsbelastningen

5.2.3 Erfarenheter av mobbprogrammering

Mobbprogrammering prövades under projektets initiala fas som ett verktyg för att snabbt få hela gruppen insatt i projektets tekniska grund. Erfarenheten visade att mobbprogrammering lämpar sig väl som en introduktionsaktivitet för gemensam kunskapsuppbyggnad, men att det är svårt att upprätthålla effektivitet vid denna arbetsform över längre tid. I detta fall konstaterades att produktiviteten ökade betydligt när gruppen övergick till mindre par eller trios, där arbetsuppgifter kunde hanteras snabbare och mer fokuserat. Figur 5.7 nedan visar resultat från en intern enkät där majoriteten av gruppmedlemmarna instämde i att mobbprogrammering fungerade bäst i projektets uppstartsfas men att andra arbetsformer var mer effektiva längre fram.

Hur upplevde du mobbprogrammeringen som var innan sprinten

9 svar



Figur 5.7: Gruppens feedback om mobbprogrammering

5.2.4 Erfarenheter av gemensamma workshops

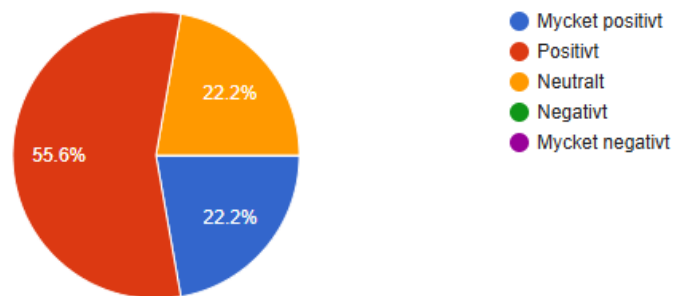
Under projektets gång planerade gruppen in återkommande workshops där medlemmarna arbetade tillsammans i gemensamma sessioner på plats. Denna arbetsform visade sig vara mycket värdefull för projektets framdrift och kvalitet. Workshops möjliggjorde snabbare kommunikation, omedelbar problemlösning och effektivare samarbete än vid arbete på di-

stans. Vid gemensamma workshops kunde frågor besvaras direkt, tekniska problem lösas i realtid och beslut fattas snabbare.

En ytterligare positiv effekt var att workshops skapade en starkare teamkänsla och ökade motivationen bland medlemmarna. Att sitta tillsammans gav en naturlig struktur åt arbetsdagen, vilket i sin tur ökade produktiviteten och bidrog till att skapa en gemensam förståelse för projektets mål och utmaningar. Figur 5.8 nedan visar resultat från en intern enkät där gruppmedlemmarna uttryckte stark uppskattning för de gemensamma workshoparna och deras betydelse för projektets kvalitet och sammanhållning.

Vad var ditt intryck av de workshops vi hade?

9 responses



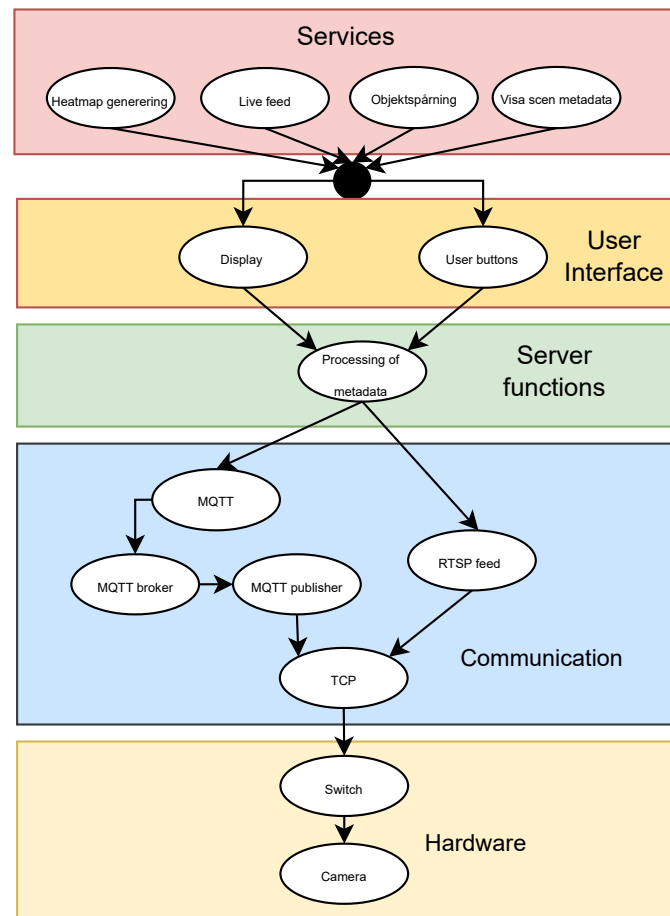
Figur 5.8: Gruppens feedback om workshops

5.3 Systemanatominns påverkan

Systemanatomien skapades i början av projektet som ett verktyg för att få en överblick över systemets olika funktioner och deras beroenden. Systemanatomien användes inte löpande under utvecklingsarbetet då gruppen redan initialt såg brister med flexibiliteten i det tillvägagångssättet. Den bidrog dock ändå till en gemensam initial förståelse för systemets struktur och prioriteringar, särskilt för de som var primärt involverade i att skriva systemanatomien.

5.3.1 Framtagningen av Systemanatomien

Framtagningen av systemanatomien genomfördes av två gruppmedlemmar i samråd med projektgruppen. Genom brainstorming och diskussioner identifierades de viktigaste funktionerna och deras beroenden, vilka organiserades i ett anatomidiagram. Fokus låg på att snabbt skapa en gemensam bild av hur systemets komponenter hängde samman för att kunna påbörja utvecklingsarbetet från ett mer strukturerat utgångsläge. Detta resulterade i systemanatomien som ses i figur 5.9.



Figur 5.9: Systematomin för projektet EagleEye

5.3.2 Effekter av systematomin på utvecklingsarbetet

Implementationen av en systemanatomy hade flera effekter på projektets utvecklingsarbete.

5.3.2.1 Stöd för tidig förståelse

Arbetet med systematomin hjälpte projektgruppen att tidigt skapa en gemensam modell av systemets huvudsakliga funktioner och beroenden, vilket underlättade uppstarten av utvecklingsarbetet.

5.3.2.2 Tydligare prioriteringar i början

Genom att identifiera vilka funktioner som var grundläggande och vilka som byggde på andra kunde utvecklingsarbetet i projektets inledningsfas planeras på ett mer strukturerat sätt.

5.3.2.3 Begränsad praktisk användning under projektet

Efter att den initiala planen var klar användes systemanatomin i begränsad omfattning i det dagliga arbetet. Fokus låg istället på att följa de mer flexibla instruktionerna som bestämdes på sprintmötena. Systemanatomin användes under senare utveckling främst som återblick till strukturen av systemet.

5.3.2.4 Erfarenhet av att använda systemanatomier

Gruppen fick en konkret erfarenhet av att skapa och använda en systemanatom, vilket kan vara värdefullt i framtida större projekt där kontinuerlig användning av en systemanatom kan spela en större roll.

5.3.2.5 Sammanfattning

Sammanfattningsvis fungerade systemanatomin främst som ett stöd i projektets inledande fas för att skapa struktur och gemensam förståelse, snarare än som ett verktyg som aktivt styrde utvecklingsarbetet hela vägen till leverans.

5.4 Förbättring av sprintplanering

För att effektivt balansera utveckling, testning och dokumentation inom varje iteration har projektgruppen arbetat aktivt med att kontinuerligt anpassa och förbättra sprintplaneringen under projektets gång. Initialt delades projekttiden in i tvåveckorssprintar med planering och uppföljning i planeringsverktyget Jira, där arbetsuppgifter organiserades med hjälp av en Kanban-tavla. Vid varje sprintstart hölls ett planeringsmöte där nya uppgifter delades ut, och varje sprint avslutades med en utvärdering där genomförda arbeten dokumenterades och lärdomar identifierades. Arbetsprocessen inspirerades delvis av principer från utvecklingsmetoden agil programmering, inklusive parprogrammering och frekventa kodgranskningar, för att säkerställa hög kodkvalitet och snabb återkoppling.

För att optimera sprintplaneringen samlades feedback in kontinuerligt under projektets gång via både utvärderingsmöten och enkäter som skickades ut efter sprintar. Enkäterna var omfattande och innefattade ett brett omfång av frågor om olika aspekter av sprintarna. Enkätsvaren samt punkter som togs upp på utvärderingsmöten var det som låg till grund för hur gruppen sedan valde att förändra arbetet i sprintarna till nästa gång.

5.4.1 Anpassningar och förbättringar under projektets gång

Under projektets inledande fas identifierades ett behov av att först gemensamt bygga upp en grundläggande förståelse för systemets struktur. Därför avvek gruppen tillfälligt från den initialt planerade strukturen med en till tvåveckorssprintarna och inledde arbetet med en intensiv sjudagarssprint där hela gruppen deltog i mobbprogrammering. Genom detta arbetssätt fick samtliga medlemmar en god inblick i projektets tekniska fundament innan utvecklingsarbetet delades upp i mindre enheter.

Efter återgång till det planerade arbetssättet, med parprogrammeringar och sprintmöte, fortsattes förbättringen. En av ändringarna som skedde var då att gruppernas storlek ökades från två till tre personer för att förbättra intern kommunikation och sprida arbetsbelastningen mer effektivt. Samtidigt förlängdes sprintarnas längd något för att ge gruppen mer tid för varje arbetsmoment samt skapa bättre utrymme för kontinuerlig feedback och ökad flexibilitet i arbetet. Mot slutet av projektet blev gruppens arbetssätt ännu mer flexibelt, där Kanban-tavlan användes aktivt för att snabbt fördela nya uppgifter och anpassa arbetsbördan efter aktuella behov.

5.4.2 Effekter av förbättrad sprintplanering

De kontinuerliga anpassningarna av sprintplanering bidrog till ett mer fokuserat och jämnt arbetsflöde i projektet. Gruppen hade tydlig ansvarsfördelning, täta avstämningar och ständig uppdatering av Kanban-flödet. Vidare hade gruppen bra flexibilitet och bra fördelning av uppgifter mellan gruppmedlemmarna. Slutligen slutfördes projektet enligt tidsplanen med samtliga kundkrav uppfyllda med hög kvalitet.



6 Diskussion

I det här kapitlet diskuteras projektets resultat och de metoder som använts under arbetets gång.

6.1 Resultat

Detta avsnitt reflekterar över projektets resultat med fokus på hur väl dessa motsvarar de ursprungliga målen och kraven. Styrkor och svagheter i den slutliga implementationen analyseras, och de valda tekniska och metodmässiga lösningarnas konsekvenser diskuteras. Avsnittet diskuterar även centrala lärdomar samt insikter som uppkommit under utvecklingsarbetet.

6.1.1 Systemanatomi

Syftet med systemanatomin var att skapa en gemensam förståelse inom gruppen kring projektets mål samt att fungera som ett verktyg för att ta fram arkitekturdokumentet [19]. En gemensam syn på projektets mål är en avgörande faktor för att ett projekt ska lyckas [20]. Detta blir särskilt viktigt i större och mer komplexa system, där agila metoder används. I sådana sammanhang kan en systemanatomi underlätta kommunikationen mellan gruppmedlemmarna och mellan gruppen och kunden [21].

Gruppen hade kunnat använda systemanatomin mer effektivt genom att följa Taxéns rekommenderade workshopmetodik [21]. Syftet med denna metodik är att involvera hela projektgruppen för att samla in så många perspektiv och synpunkter som möjligt. I projektet utvecklades dock systemanatomin istället av enbart projektets arkitekt och utvecklingsledare, vilket begränsade dess värde. Eftersom systemanatomin inte reviderades regelbundet tappade den dessutom sin funktion som gemensam referensram för projektgruppen, vilket enligt både Cerny och Taxén är avgörande för att skapa en delad förståelse [19], [21].

För att maximera nyttan av systemanatomin borde gruppen ha genomfört workshopen tillsammans och följt upp dokumentet löpande, tills dess att andra dokument, som arkitekturbeskrivningen, tog över dess roll. Trots bristerna gav systemanatomin ändå ett viktigt initialt stöd för att förstå projektet och låg till grund för det vidare arbetet med arkitekturen.

6.1.2 Alternativa implementationssätt

Valet av programmeringsspråk och ramverk för frontend och backend var specificerat av kunden och utgjorde därmed fasta krav för projektet. Trots dessa krav fanns det flera alternativa sätt att implementera systemet på, med möjlighet till optimering av prestanda och mängd kod. Språken som valdes, *Python* och *TypeScript* med *React*, gav stora fördelar i utvecklingen av projektet på grund av det stora utbudet av bibliotek som hjälpte med programmeringen.

En central fråga i utvecklingen var användningen av det externa programmet *RTSPto-WebRTC* [4]. Programmets syfte var att konvertera RTSP-strömmarna från kamerorna till WebRTC-strömmar i backend, vilket det också lyckades med. Dock var programmet skrivet i programmeringsspråket Go, vilket innebar att ytterligare krav ställdes på systemets driftmiljö.

Ett alternativ till denna lösning hade varit att använda React-biblioteket *media-stream-library-js*, som möjliggör direkt uppspelning av kamerans vy i frontend utan att behöva gå via en separat konvertering i backend. Detta skulle potentiellt förenkla arkitekturen och minska beroenden, men detta alternativ valdes inte av gruppen.

Även om kunden inte ställde några specifika krav på att en frontend skulle vara implementerad som en webbapplikation, angavs det att om det görs, skulle React med TypeScript användas. Ett alternativt tillvägagångssätt hade varit att skapa en fristående applikation med ett grafiskt användargränssnitt i Python som hanterar både backend och användargränssnittet i ett och samma system. En sådan implementation hade kunnat vara enklare att utveckla, men gruppen valde i stället att separera backend med en webbaserad frontend. Det möjliggjorde en tydligare systemstruktur och ett mer läsbart, underhållsbart och användarvänligt gränssnitt.

Slutligen kunde frontendens CSS optimeras med hjälp av ramverk som Tailwind CSS. Genom att använda Tailwinds konfigurationsmöjligheter kan koden struktureras modulärt, vilket leder till mer läsbara och underhållsvänliga filer. Detta skulle ha fört med sig att de CSS-filer som gruppen implementerade med vanlig CSS hade blivit enklare att förvalta och därigenom ökat utvecklingseffektiviteten utan att kompromissa med kundens krav.

6.1.3 Viktiga lärdomar från resultatet

En viktig lärdom från utvecklingen av EagleEye-systemet berör hantering av realtidsvideostömmar från kamerorna och dess påverkan på systemets prestanda. Under projektets tidigare fas användes verktyget FFmpeg direkt i backend-servern för att omvandla RTSP-strömmar till strömningsformatet HLS för uppspelning i frontend. Detta gjordes eftersom HLS stöds internt av webbläsare, till skillnad från RTSP. Metoden resulterade i fördröjningar på flera sekunder, vilket ansågs oacceptabelt för ett system med krav på realtidsvisualisering.

För att adressera problemet implementerades en separat RTSPtoWebRTC-server, baserat på ett öppet källkods verktyg skrivet i Go [4]. Denna lösning konverterade RTSP-strömmarna till WebRTC med lägre latens än den tidigare konverteringen. Det möjliggjorde att videostömmen kunde spelas upp i frontend med mindre än en sekunds fördröjning jämfört med den tidigare implementationen som hade flera sekunders fördröjning. Detta uppfyllde kundens krav på låg latens och förbättrade användarupplevelsen av systemet.

Lärdomen är att valet av kommunikationsprotokoll är avgörande för systemets prestanda. WebRTC är specifikt utformat för realtidskommunikation medan HLS är mer lämpat för konvertering av förinspelade videofilmer. En nackdel med WebRTC-implementationen är dock att den medför ett ytterligare beroende och en till installation för användaren, eftersom RTSP-till-WebRTC-servern är baserad på Go. Trots det vägde fördelarna med låg latens tyngre med tanke på att Axis Communications har som mål att använda EagleEye för demonstrationer i realtid.

En annan betydande lärdom rör hanteringen av geokoordinater för kamerornas positionering vilket var en central del för att visa kamerornas metadatafunktioner. Bristen på speci-

aliserade verktyg för att mäta exakta geokoordinater, kombinerat med att kamerorna placerades inomhus där GPS-signaler är opålitliga skapade betydande utmaningar. Gruppen var begränsad till att använda telefonbaserad GPS, vilket gav otillförlitliga resultat för inomhusmiljöer.

För att överkomma detta problem antogs ett nytt tillvägagångssätt. I stället för att använda de faktiska koordinaterna så baserar projektet sig på relativa koordinater, där ett simulerat rum definierades i geokoordinater och kamerorna tilldelades relativa positioner i detta rum. Denna lösning möjliggjorde funktionalitet för rörelseanalys och visualisering av metadata genom heatmaps, men begränsade systemets förmåga att demonstrera verkliga geopositioner.

Lärdomen av detta är att noggrann geopositionering kräver tillgång till avancerade mätverktyg, speciellt för inomhusmiljöer då de kan kräva alternativa metoder, som användningen av manuellt definierade koordinatsystem för att kompensera för de tekniska begränsningarna. Denna anpassning var avgörande för att uppfylla projektets demonstrationssyfte men understryker behovet av en förbättrad hårdvara och metodik för framtida implementationer. En ytterligare lärdom är att homografibaserad translation är en effektiv metod för att översätta objektpositioner från kameravy till kartvy som komplement till den geokoordinatsbaserade lösningen. Genom att använda mitten av den nedre kanten av ett objekts bounding box (motsvarande fötternas position) och mappa fyra punkter från en kameraskärmdump till motsvarande punkter på kartvyn, kunde gruppen noggrant transformera objektpositioner utan att behöva ange kamerans installationshöjd, riktning eller exakta geokoordinater.

Denna metod visade sig robust och förenklade konfigurationen, särskilt i inomhusmiljöer där geokoordinater var svåra att erhålla. Axis communications prioriterade geokoordinater för att demonstrera deras nya kamerafunktioner men uppmuntrade att behålla homografibaserade metoden för jämförelse. Jämförelsen visade att homografin erbjöd snabbare och mindre resurskrävande konfiguration, medan geokoordinater gav potential för skalbarhet i utomhusmiljöer med exakta positioner.

Lärdomen är att situationsanpassa metoder, som homografi, kan vara effektiva för att lösa tekniska begränsningar, men att kundens strategiska mål (att visa nya funktioner) måste vägas in i tekniska beslut. Detta understryker vikten av flexibilitet i systemdesign för att balansera prestanda och kundnytta.

6.1.4 Analys av kundnytta och vidareutveckling

I detta avsnitt kommer projektet EagleEye analyseras utifrån kundnyttan och framtida vidareutveckling.

6.1.4.1 Kundnytta

Ett huvudsakligt sätt EagleEye har skapat värde för kunden, Axis Communications, är att kombinera realtidsövervakning med ett visualiseringsverktyg. Systemet fyller en värdefull roll i säkerhetsrelaterade användningsområden, där snabba reaktioner är väsentliga, genom systemets låga latens. Vidare gav heatmap funktionaliteten ytterligare värde till kunden genom möjligheten till rörelseanalys. Utöver det skapades värde för kunden genom larmfunktionaliteten för ytterligare säkerhet. Slutligen fann kunden värde genom projektets testning av deras nya teknik, geokoordinater. Kunden fick insyn i hur nya produkter eller verktyg hade kunnat få användning av geokoordinater.

6.1.4.2 Vidareutveckling

Vid överlämnandet av projektet hade EagleEye en del steg kvar för att värdet för kunden skulle uppfyllas fullt ut.

En integration med kundens befintliga system och processer är ett av de första stegen projektet hade behövt tagit. I samband med en sådan integration hade dokumentation för

flera olika användargrupper behövs förbättras. Vidare hade en förbättrad användarvänlighet, med en mer utvecklad frontend, skapat ytterligare värde för kunden. Kunden har visat ett intresse att förminska svårigheten för konfiguration av systemet. Med en mer flexibel frontend, som hade kunnat konfigurera kamerornas geokoordinater, hade ytterligare värde kunna skapas för kunden.

Kunden visade även stort intresse av systemets förmåga att identifiera ifall två identifierade objekt egentligen är ett och samma objekt, vilket kan hända ifall två olika kameror ser samma objekt från två olika vyer. En vidareutveckling skulle därför vara att förbättra igenkänningen av samma objekt. Genom en mer robust identifieringsalgoritm kan spårningen av rörelser bli alltmer tillförlitlig.

Sammanfattningsvis finns det flera vidareutvecklingsområden av systemet inom bland annat skalbarhet, flexibilitet och intelligens.

6.2 Metod

I detta avsnitt diskuteras de metoder som användes under utvecklingen av EagleEye. Genom att analysera dessa metoder identifieras styrkor, svagheter och förslag på förbättringar.

6.2.1 Metod för att samla erfarenheter

I denna del kommer metoden som användes för att samla erfarenheter diskuteras.

6.2.1.1 Sprintenkäter

En metod som användes för att samla erfarenhet var sprintenkäter. Mot slutet av varje tvåveckorssprint skickades enkäter ut och gav alla gruppmedlemmar möjlighet att utvärdera sprinten. Resultaten sammanställdes sedan inför nästa sprintmöte där teamet diskuterade åsikter och förslag till förbättringar. Enkäterna visade sig vara ett bra sätt att uppmärksamma problem som gruppmedlemmar upplevt med sprinten och utvecklingsprocessen. Eftersom sprintenkäterna var anonyma kunde gruppmedlemmarna uttrycka sina åsikter fritt, vilket främjade öppet tänkande och att alla fick göra sin röst hörd. Dessutom fungerade enkäterna som en katalysator till förbättring, eftersom de uppmuntrade till aktiv reflektion om processens brister och förbättringsmöjligheter.

En potentiell förbättring hade varit att ha tätare enkätutskick och därmed ha möjlighet att upptäcka och åtgärda upplevda problem tidigare. Däremot kan det argumenteras för att problem redan uppmärksammas naturligt när gruppmedlemmar pratar med varandra på möten eller över kommunikationskanaler. Om tätare enkätutskick ska användas i framtida projekt borde enkäterna vara kortare för att kompensera för extra arbetet som uppstår.

6.2.1.2 Kundmöten

En ytterligare metod för insamling av erfarenheter var kundmöten med Axis Communications. Representerarna från Axis Communications var erfarna mjukvaruutvecklare med god kunskap om kamerorna och kunde därför ge värdefulla insikter under mötena. Kundmötena var även en plattform för gruppen att presentera prototyper och diskutera de förväntningar som Axis Communications hade på slutprodukten.

Ett problem som upplevdes var att kunden gav gruppen till synes fria tyglar i utformningen av projektet i helhet. Detta ledde till ovisshet om vad kunden hade för faktiska förväntningar på projektet. Bristen på riktlinjer resulterade i att gruppen till en början prioriterade fel och designade oönskad funktionalitet, som till exempel ett användarsystem med inloggningsfunktion. Detta kunde dock åtgärdas genom att på senare kundmöten ställa mer specifika frågor om vad kunden faktiskt önskade sig. Det skapade en tydligare målbild.

En viktig lärdom var att tidigt undersöka de krav som kunden har genom att ställa specifika frågor kring systemdesign under kundmöten. Detta minskar risken för missförstånd och att oönskad funktionalitet utvecklas.

6.2.1.3Handledarmöten

En tredje metod för erfarenhetssamling var handledarmöten som hölls varje vecka med alla gruppmedlemmar närvarande. Under mötena erbjöds vägledning i projektet och det gavs möjlighet att ställa frågor till handledaren om projektets akademiska krav. Handledaren var tekniskt kunnig och hade tidigare erfarenhet av liknande projektarbeten. Som vägledare i själva utvecklingsprocessen var handledaren en värdefull resurs. Däremot var handledarens insikt i specifika tekniska detaljer begränsad, delvis på grund av ett sekretessavtal som gruppmedlemmarna hade tecknat gentemot Axis Communications. Trots det kunde generella råd kring systemdesign ges.

En möjlig förbättring hade varit att inkludera handledaren i sekretessavtalet för att möjliggöra en mer öppen diskussion om projektet. Gruppen upplevde att handledaren inte involverade sig så mycket som den potentiellt hade kunnat på grund av sekretessavtalet. Handledarens försiktighet, förmodligen orsakad av osäkerhet om gränserna för vad som fick delges, ledde i vissa fall till mindre engagemang i tekniska diskussioner än vad som hade varit möjligt. Ytterligare en begränsning som gruppen upplevde var att handledaren inte kunde svara på ett flertal frågor om kursens innehåll och dess akademiska krav. Det berodde i sin tur på bristfällig information från examinator till handledare.

Som effekt fanns det en del tekniska och skrivna moment som behövdes förändras eller omtänkas på grund av mindre information. Vissa bitar av projektet stannades upp i osäkerhet och påverkade projektet negativt i produktivitet.

6.2.2 Arbetsmetodik

Detta avsnitt diskuterar de arbetsmetodikerna som gruppen använde inom projektets ramar.

6.2.2.1 Scrum-sprintar

Projektet organiserades i tvåveckorssprintar, där gruppen delades in i tre grupper om tre personer. Upplägget upplevdes av gruppen som effektivt, eftersom det möjliggjorde fokuserat arbete och regelbunden utvärdering. Ett problem uppstod under den första sprinten, där vissa gruppmedlemmar upplevde att deras tilldelade uppgifter var beroende av andra gruppers arbete. Detta ledde till att två grupper försökte lösa samma problem oberoende av varandra och utan något samarbete. Det resulterade i redundant arbete när en grupp slutförde uppgiften först. Problemet kunde identifieras i ett tidigt skede tack vare användandet av sprintenkäter. Sprintenkäterna användes för att försöka förbättra utvecklingsprocessen. Inför efterföljande sprintar justerades planeringen för att undvika beroenden mellan grupper och därmed förbättra effektiviteten. För framtida projekt kan arbetsberoenden i sprintar hållas i åtanke.

Den agila metodikens iterativa natur, med sprintenkäter som verktyg, visade sig vara bra på att snabbt identifiera problem så att förbättringar sedan kan göras. En förbättring som gjorde var att ändra storleken på sprintgrupperna från två till tre. Den förändringen förbättrade projektet då de mindre grupperna kunde ta ann större uppgifter.

6.2.2.2 Workshops

Workshops användes för informationsdelning, särskilt under uppstartsfasen när gruppens kunskaper var begränsade. I detta skede var workshops värdefulla och gav hela gruppen en grundläggande kompetens. Planerade workshops, där specifika demonstrationer var förberedda, visade sig vara särskilt effektiva för att sprida kunskap. Ett exempel var när konfigu-

reringen av kamerorna demonstrerades i helgrupp. En erfarenhet från detta är att workshops är ett effektivt sätt att fördela kunskap och information mellan medlemmar

6.2.2.3 Versionshantering

Gruppen använde sig av en versionshantering baserad på funktionsgrenar där varje sprintgrupp arbetade på en egen gren för utveckling av ny funktionalitet. Strategin upplevdes som effektiv eftersom den möjliggjorde parallell utveckling, vilket tillät sprintgrupper att arbeta oberoende av varandra utan att det uppstod konflikter i koden. På grund av gruppens storlek hade koordinering blivit svår utan denna uppdelning. En annan fördel var att det alltid existerade en stabil huvudgren eftersom endast fungerande kod implementerades i den.

En utmaning som upplevdes av samtliga gruppmedlemmar var sammanslagningen av funktionsgrenar till huvudgrenen. En förbättringsmöjlighet till framtida projekt är att använda sig av så kallade merge-förfrågningar samt att utse en person eller grupp som ansvarar för granskning och implementation av dessa. En sådan strategi testades aldrig, men hade potentiellt effektiviserat integration av ny kod. Ytterligare en förbättringsmöjlighet hade varit att implementera automatiska tester som utförs på varje ny merge-förfrågning.

6.2.2.4 Kanban

En Kanban-tavla kan användas för att ta bort flaskhalsar och att minska onödig väntetid för att öka teamets effektivitet [22]. Att använda en Kanban-tavla i samband med en agil metodik och en backlog kan effektivitet av gruppen ökas ständigt. Det är en adaptiv och ständigt utvecklande tavla med backloggar.

Gruppen har använt en Kanban-tavla i Jira för att hålla koll på hur arbetet utvecklas och går framåt. Det har varit ett hjälpsamt verktyg för att skapa struktur och översikt. Dock kan uppsättningen och användningen optimeras. Exempelvis saknades en begränsning av mängden arbete i pågående fas (WIP), vilket ledde till att för många uppgifter hanterades samtidigt. Detta resulterade i delvis överflöde av parallellisering. Hantering av mängden arbete som är i WIP är en av huvudprinciperna av en Kanban-tavla och med att gruppen la mycket där samtidigt gick emot principen och minskade effektiviteten [23].

För att motverka detta bör tydliga gränser för Kanban-tavlans användning skapats.

En annan förbättring var att optimera Kanban-tavlans backlogg. Enligt Sedano et al. bör en backlogg innehålla konkreta resultat, tydliga prioriteringar, uppskattad tidsåtgång och kontinuerlig uppdatering för att maximera värdet för projektet [24]. Genom att tillämpa denna backloggen hade gruppen kunnat uppnå högre effektivitet i den agila utvecklingen.

6.2.3 Reviderade roller

Rollerna som tilldelas i början av projektets gång bidrog till en grundläggande struktur och ansvarsfördelning inom gruppen. I de första stegen av projektets gång var det därför tydligt vem som förväntades ha ansvar för vad. Men som beskrivs i avsnitt 5.2.1 och 5.2.4 förändrades arbetssättet mot en mer flexibel och dynamisk modell där medlemmar tog sig an uppgifter över rollgränserna.

Ett exempel på rollförändring var arkitekten. Rollen innebar ett ansvar för övergripande teknisk struktur samt arkitekturmässiga svar, se ansvar beskrivning i avsnitt 1.6.1. Under projektets gång krävdes en större ordning mellan delsystemen, speciellt vid implementation av konvertering av koordinater 5.1.3 samt visualisering av rörelse i realtid 5.1.4. Som reaktion på dessa implementationer engagerade sig andra gruppmedlemmar i arkitekтуella diskussioner.

Ett dynamiskt arbetssätt där flera tar ansvar kan visa sig behövas även när en grupp har formella roller utdelade. I fallet med arkitekturen skapades ett samarbete, istället för en isolerad specialroll, vilket hjälpt gruppen att hantera komplexiteten i projektet. Sammanfattningsvis tyder gruppens erfarenheter på att projekt vinner på att tilldela mer flexibla roller.

6.2.4 Testning

De testresultat som presenterades visar att projektet hade fokus på att kvalitetssäkra koden och systemets delar. Enhetstesterna för `Mapmanager` och `Objectmanager` identifierade tidigt mindre fel samt gränsfall, som hade kunnat orsakat mycket skada om det integrerades utan att åtgärdats. Vilket också stärkte gruppens förtroende för att komponenterna fungerade tillfredsställande. Även om teststäckningen för dessa komponenterna var god hade gruppen kunnat använda sig av fler integrationstester för att testa komponenterna efter integrering för att ge gruppen ytterligare trygghet i stabiliteten av systemet.

Latensmätningen av kamerasytemet visar just testning kan påverka viktiga tekniska beslut. Initialt visade sig HLS-formatet för videoöverföring ha för hög latens för systemets syfte, vilket i sin tur ledde till en välinformerad övergång till WebRTC. Detta visar på nyttan av att mäta prestanda tidigt i utvecklingen. Att få reda på denna informationen innan gruppen implementerat HLS i helheten sparade gruppen mycket tid som hade gått åt till onödig utveckling annars. En sak som hade kunnat göras bättre är inte använda manuell mätning. Detta kan ge varierande resultat men också introducerar det mänskliga felet. Istället hade tidsstämplat kunnat användas för att köra tester och mäta latensen för videoöverföringen. Genom detta tillvägagångssätt erhöles mer precisa och konsekventa testresultat.

Testningen av larmfunktionen visade inga fel, vilket var väntat med tanke på dess enkla logik. Här hade testningen kunnat utökas med andra komponenter, för att sedan testa en större del av systemet och se hur de samverkar med andra delar.

6.2.5 Resurser

I detta avsnitt kommer det diskuteras hur resurserna har påverkat projektet.

6.2.5.1 Tekniska

De tekniska resurserna har haft en tydlig inverkan på projektets genomförande. Den mest påverkande faktorn var tillgången till hårdvara. Gruppen hade endast tillgång till en nätverksswitch, vilket begränsade möjligheten att koppla upp flera enheter samtidigt. Dessutom hade inte alla tillgång till en router med "Power over Ethernet" (PoE), vilket är en nödvändighet för att driva kamerorna. Bristen på tillräcklig hårdvara, som till exempel kameror och ethernetkablar, gjorde det svårt att parallellisera arbetet mellan olika gruppmedlemmar. Det innebar att vissa delar av projektet fick skjutas upp, och att gruppen behövde arbeta med simulerad data för att testa och utveckla.

Utöver hårdvaruresurserna har gruppens tekniska kompetens påverkat projektets gång. Majoriteten av gruppen hade aldrig jobbat med React, MQTT och REST (flask endpoint). Detta ledde till att en del av tiden gick åt att åtgärda detta, både genom workshops och individuell inläring. Detta tog upp en del av den totala tilldelade tiden men samtidigt bidrog den till en mer kunnig grupp och jämnfördelad kunskapsbörda.

6.2.5.2 Mjuka

Utöver de tekniska resurserna spelade även mjuka resurser en roll i projektets resultat. Den avgörande mjuka resursen tid var något gruppen var begränsad av. Varje gruppmedlem var tilldelad 400 timmar av arbete. Vidare gick kurser parallellt med projektet. Projektet anpassades utifrån dessa nämnda förutsättningar genom en agil metodik där arbetsuppgifter kunde omfördelas vid behov 5.2.1. På så sätt kunde ett jämnt arbetsflöde upprätthållas även när vissa gruppmedlemmar var begränsad med tillgänglighet.

En annan aspekt var motivation och engagemang. Att arbeta med en verklig kund, Axis Communications, ökade den upplevda vikten i projektet. Däremot varierade medlemmarnas tillgänglighet för workshops och annat samarbete, något som visade sig ha stor påverkan på

framdriften. Som beskrivits i avsnittet 5.2.4 visade sig workshops, med hela gruppen deltagande, mer produktiva och ge bättre lösningar.

Projektet visar tydligt hur begränsade mjuka resurser, som tid och tillgänglighet, kan hanteras med metodiska anpassningar och samarbete.

6.2.6 Alternativa utvecklingsmetoder

Inom kursens ram bestämdes det att en agil-metodik skulle implementeras i projektets gång. Dock finns det alternativ som skulle kunna användas. Vidare fanns det flera metoder inom de agila ramverken som skulle kunna ha användas av gruppen.

En annan vanlig planeringsmetod är användningen av Gantt-scheman, där aktivitet visualiseras över tid med en kritisk väg. Detta skulle ge ett tydlig översikt för gruppen och ge möjligheten att parallellisera stora mängder av arbetet. I projekt med starkt definierade milstolpar och uppgifter med tydliga start och slutdatum passar Gantt-scheman bra.

Även inom agila-metodiken skulle metoder som "Extreme Programming"(XP) kunna användas istället för Scrum för att uppnå samma effekt som gruppen hade redan gjort.

6.3 Arbetet i ett vidare sammanhang

Detta avsnitt analyserar projektets effekter i ett bredare sammanhang. Analysen utgår från modellen enligt Becker et al., *"Requirements: The Key to Sustainability"* (SusAF) [25]. De dimensioner som inkluderas är: sociala, ekonomiska, tekniska, miljömässiga och individuella. De SusAF-diagram som används är modifierade versioner av de som presenteras i *"Requirements: The Key to Sustainability"*.

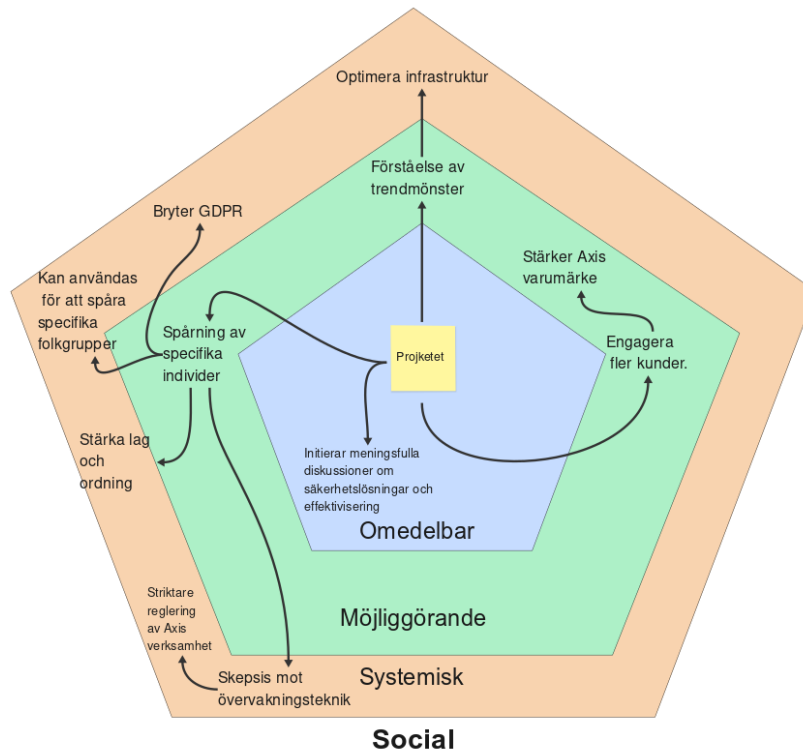
6.3.1 Social dimension

När ett projekt utvecklar ett verktyg kopplat till bevakning och säkerhet väcks ett antal sociala frågor. Figur 6.1 gör en visualisering på flera av dessa perspektiv. Verktøget som utvecklats inom projektet har potentialen att användas för att spåra individer, vilket kan vara motiverat ur en säkerhetssynpunkt, men kan även väcka frågor om integritet och etik. Användningen av objektigenkänning och objektspårning innebär en risk för att tekniken kan missbrukas, särskilt om den vidareutvecklas utanför den avsedda syftet.

Samtidigt kan projektet bidra till ett ökat trygghet inom samhällen och sociala gäng. Genom att verktygets möjlighet att visualisera rörelser och använda larmzoner kan verktyget identifiera potentiella hot, detta leder till ett mer effektiv och proaktiv övervakning istället för reaktiv. Detta kan öka trygghetskänslan för vardagliga aktiviteter.

Användningen av heatmaps och objektspårning kan även detektera trendmönster från folk. Detta är något Axis Communications redan har gjort i Japan för att optimera stadsplaneringen för folket, detta gjordes med liknande teknik men med personräkning [26]. Exakt som i Japan kan verktyget användas för att se trendmönster och optimera vägar för människor. Detta tas även upp från Collini et al, hur användningen av termiska kameror kan användas för att se trendrörelser och hur det kan användas för att optimera stadsplanering [27].

Slutligen öppnar verktygets upp för bereda samhällsdiskussioner kring balansen mellan säkerhet, integritet, samt vilka riktlinjer och regelverk som bör finnas för teknologins användning. Detta är en av de viktiga sociala effekterna från projektet.



Figur 6.1: SusAD diagram för den sociala dimensionen

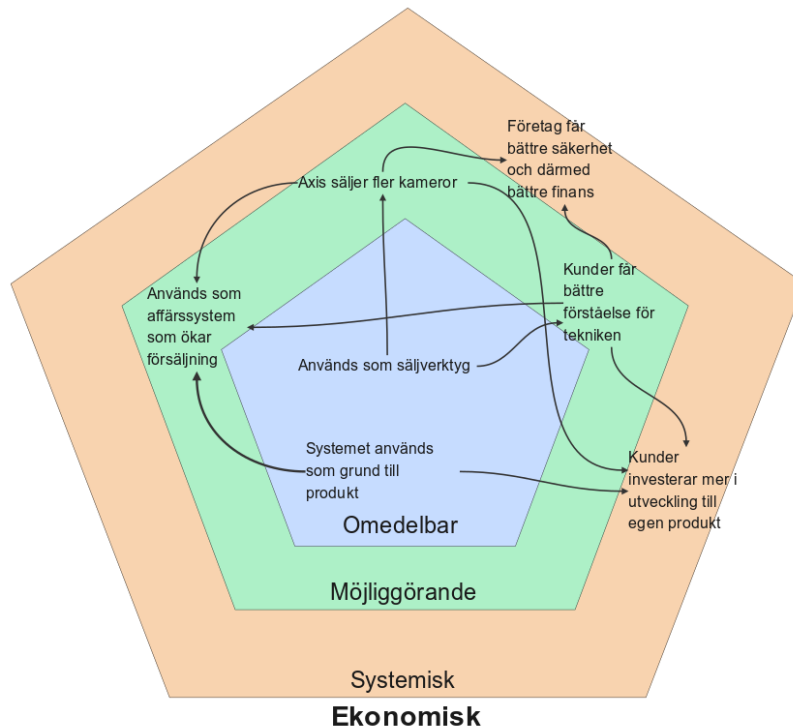
6.3.2 Ekonomisk dimension

Verktyget som utvecklades kan få ekonomiska konsekvenser som visas i figur 6.2. Lösningen som användes kan bli ett incitament för att omfördela resurser inom säkerhetsbranschen särskilt mot mer tekniska lösningar. Även om verktyget är tillgängligt som öppen källkod, krävs det viss specifik hårdvara för att det ska fungera. Utan kameror som ger scen-metadata fungerar inte verktyget. Kunden Axis Communications är de som har kameror med just den här teknologin för tillfället.

Detta kräver att andra aktörer behöver investera i hårdvara för att använda detta verktyget, vilket Axis Communications gynnas av. Samtidigt kan detta vara ett inspirerande moment där fler investerar i sina egna lösningar som leder till fler konkurrenter. Generellt sätt motiverar användningen av verktyget att fler kameror som hjälper Axis Communications sälja mer.

På längre sikt kan verktyget påverka vilka aktörer som har störst inflytande inom analys- och säkerhetslösningar, vilket kan leda till förändringar i hur marknaden är uppbyggd.

Slutligen är detta endast ett verktyg som kräver vidareutveckling för att bli en produkt. Det betyder att det finns en börda för nästa person som jobbar på projektet.

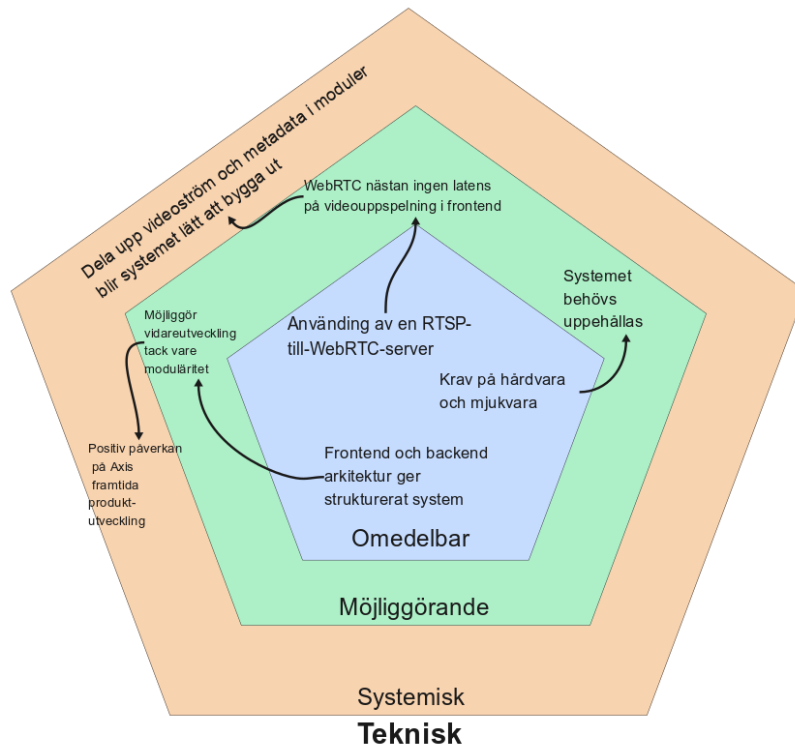


Figur 6.2: SusAD diagram för den ekonomisk dimensionen

6.3.3 Teknisk dimension

De tekniska beslut som fattas under projektets gång bidrar till att teknisk skuld byggs upp. Enligt figur 6.3 framgår att valet att använda det externa programmet RTSPtoWebRTC medför ett beroende av att detta program fortsatt underhålls. Detta beroende har både för och nackdelar. Om programmet skulle sluta underhållas, faller ansvaret på projektgruppen att hitta en alternativ lösning. Samtidigt slipper gruppen utveckla en egen lösning från grunden, vilket minskar utvecklingsbördan. De krav som ställs på både mjukvara och hårdvara innebär därmed ett fortsatt behov av att verktyget förblir tillgängligt och fungerar korrekt.

Tack vare projektets modulära uppbyggnad kan framtida utvecklare enkelt ta vid och vidareutveckla systemet, förutsatt att dokumentationen är tillräcklig. Det innebär att Axis Communications kan välja att ta bort vissa delar och vidareutveckla andra utan att det påverkar helheten negativt.



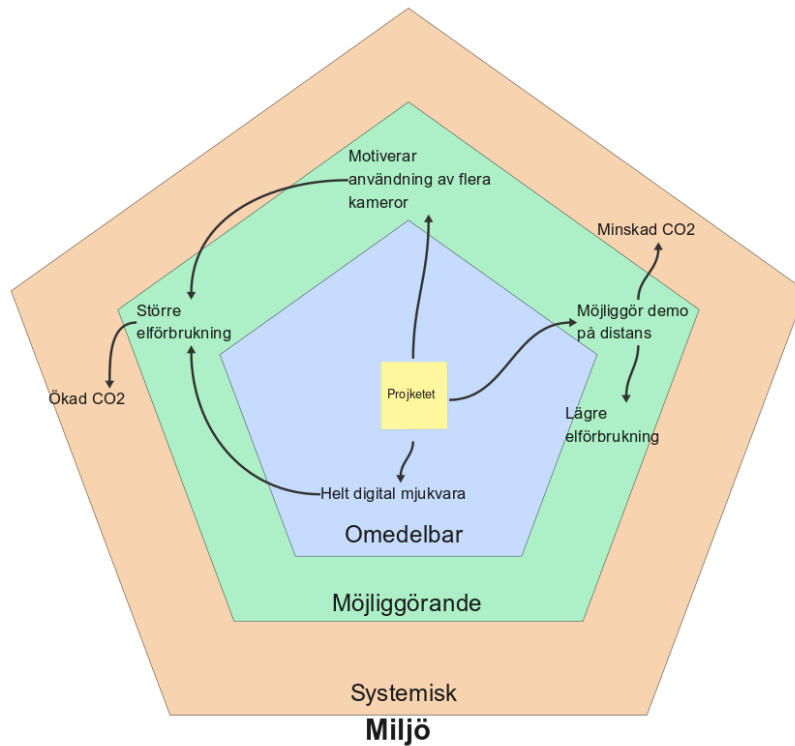
Figur 6.3: SusAD diagram för den tekniska dimensionen

6.3.4 Miljömässig dimension

Utifrån figuren 6.4 kan projektet analyseras i ett hållbarhetsperspektiv från omedelbara, möjliggörande och systemiska effekter.

Genom projektets digitala lösning möjliggör det demonstrationer och arbete på distans vilket i sin tur skulle leda till en minskat koldioxidutsläpp och elförbrukning. Effekterna kan ses i de yttersta lagerna i figur 6.4 och tyder på att projektet ha en positiv effekt inom hållbar utveckling.

Samtidigt finns det negativa effekter, till exempel motiverar projektet användning av fler kameror. Flera kameror leder i längden till en ökad elförbrukning samt till ökad koldioxidutsläpp. Projektet påverkar därför miljön både positivt och negativt.

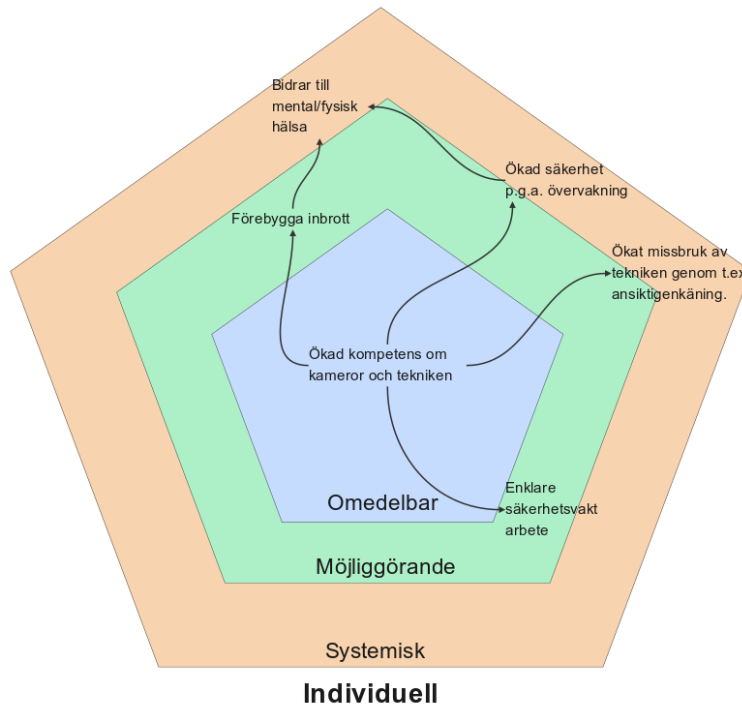


Figur 6.4: SusAD diagram för den miljömässiga dimensionen

6.3.5 Individuell dimension

I figur 6.5 visas det att projektet främjar individers trygghet och välbefinnande på flera olika sätt. Den ökade säkerheten projektet kan bidra till skulle kunna förebygga inbrott. Ett område med färre inbrott skulle därefter minska stress och rädsla hos boende och personal i närheten. Vidare skulle projektet kunna höja kompetensen kring användningen av övervakningssystem med hjälp av demonstrationer.

Däremot illustrerar även figur 6.5 att det finns en del risker som projektet kan påverka. En av de stora riskerna är att genom ökad kompetens kan det ske en ökning av missbruk av teknikerna projektet utvecklat. Som systematisk konsekvens skulle det kunna innebära integritetsproblem. Det betyder att det finns en stor vikt i riktlinjer och regelverk för hur tekniken får användas, speciellt i offentlig miljö.



Figur 6.5: SusAD diagram för den individuella dimensionen

6.3.6 Källkritik

För att säkerställa vetenskaplig trovärdighet har en analys genomförts av de referenser som används i projektet. Källorna är grupperade efter källtyperna: vetenskapliga artiklar, officiella dokumentationer, blogginlägg och metodologiska källor, och har bedömts utifrån deras trovärdighet, aktualitet, relevans och eventuella begränsningar.

6.3.6.1 Vetenskapliga artiklar

Vetenskapliga artiklar bidrar med teoretisk grund och faktabaserad data. Detta stärker rapportens tekniska och analytiska resonemang. Några vetenskapliga artiklar som används i denna rapport är [12], [27] och [23]. Dessa artiklar är aktuella (2018-2024) och berör relevanta ämnen som Git pipelines och objektspåring. Det negativa med vetenskapliga artiklar är att de ofta kan vara svåra att tillämpa i projektet då de ofta kan vara väldigt specialiserade för sitt ämne.

6.3.6.2 Officiella dokumentationer

Officiella dokumentationer erbjuder teknisk noggrannhet och praktiska insikter i verktyg och teknologier som används i projektet. Källor som dokumentation från Axis Communications [28] och dokumentation på tredjepartsverktyget RTSPtoWebRTC [4] är aktuella och kompetenta för teknologierna de beskriver. Dock kan officiell dokumentation bidra till partiskhet då det ofta är skrivet med kommersiella motiv vilket kan leda till en främjande ton.

6.3.6.3 Blogginlägg

Blogginlägg kan gynna praktiska och uppdaterade perspektiv om verktygen som används i projektet, vilket kan vara användbart för att snabbt förstå teknologierna som tillämpas. Källor som [3] om WebRTC är aktuell (2023) och assisterar i förståelsen över teknologin, men dess

icke-vetenskapliga bakgrund riskerar generalisering vilket gör sådana källor mindre pålitliga. Blogginlägg har därför endast använts som stödande material för att ge tekniska koncept ett bredare sammanhang.

6.3.6.4 Metodologiska källor

Metodologiska källor bidrar med strukturerade ramverk för projektledningen och systemutveckling, vilket ger ett bra underlag för projektets utvecklingsmetoder. Källor som [20], [21] och [9] är erkända inom agila metoder och systemanatomi. Dock är de något daterade (2006-2011), men det har inte lika stor betydelse för metodologiska källor som tekniska referenser, då teknik utvecklas i mycket högre fart.



7 Slutsats

Detta kapitel återkopplar till frågeställningarna från avsnitt 1.4 genom att ta slutsatser från gruppens erfarenheter.

7.1 Hur kan EagleEye-systemet implementeras för att skapa värde för kunden, Axis Communications?

För att skapa värde för kunden bör en utvecklingsgrupp ha nära samarbete med kunden. Det är av hög vikt att tidigt definiera tydliga krav för projektet för att säkerställa att projektgruppen vet vad kunden vill ha och vad som ska utvecklas. I en agil och iterativ arbetsprocess är det viktigt att det nära samarbetet med kunden fortsätter även under utvecklingsfasen. Det säkerställer att eventuella missförstånd kring förväntningar upptäcks i tid och att kunden blir nöjd med projektets framsteg. Genom återkommande kundmöten och demonstrationer av projektets status har kundens insikter och behov kunnat integreras direkt i utvecklingsprocessen för att skapa maximalt värde för kunden. Denna återkoppling har inte bara styrt projektets utformning utan även bidragit till att projektet utvecklats på ett sätt som underlättat vidareutveckling av system för att kunna skapa fortsatt värde åt kunden.

7.2 Vilka erfarenheter, som kan vara intressanta för framtida projekt, kan dokumenteras från EagleEye-projektet?

Under projektet samlades flera viktiga erfarenheter som kan vara värdefulla för framtida projekt. För det första visade det sig att en agil arbetsmetodik, där gruppmedlemmar har stor möjlighet att omfördela arbetsuppgifter efter behov, förbättrar effektiviteten avsevärt. Det gjorde det möjligt att anpassa arbetsbelastningen efter gruppens tillgänglighet och minska flaskhalsar.

För det andra visade användningen av tydlig och transparent dokumentation i ett gemensamt verktyg som Jira och Kanban-tavlan att det ökade ansvarstagandet och tydliggjorde arbetsfördelningen. Detta bidrog till ökad produktivitet och förbättrad arbetsmoral. En tredje erfarenhet är kopplad till slutsatsen från avsnitt 7.1. Vikten av att tidigt etablera tydliga förväntningar från kunden. I början av projektet resulterade bristen på specifika krav i ovisshet och onödig utveckling av funktioner. Regelbundna och specifika kundmöten under projek-

tets gång var avgörande för att säkerställa att utvecklingen fokuserade på kundens verkliga behov.

Dessutom visade sig workshops vara mest effektiva när de var planerade och hade tydliga mål. Under projektets senare skede blev oplanerade workshops ineffektiva och tog upp onödigt mycket tid från individuella arbetsuppgifter. Framtida projekt bör därför fokusera på noggrant planerade och målmedvetna workshops. Ytterligare en viktig lärdom var vikten av en strukturerad och anpassningsbar sprintplanering. Genom att kontinuerligt utvärdera och anpassa sprintupplägget kunde teamet säkerställa en bra balans mellan utveckling, testning och dokumentation.

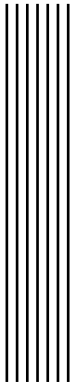
Slutligen är erfarenheten av versionshantering med funktionsgrenar och en stabil huvudgren viktig. Den möjliggjorde parallell utveckling utan kodkonflikter, men visade också att det krävs strukturerade rutiner kring merge-förfrågan och kodgranskningar för att effektivisera integrationen av nya funktioner. Dessa erfarenheter utgör sammantaget en god grund som kan öka effektivitet, kvalitet och kundvärde i framtida utvecklingsprojekt.

7.3 Vilket stöd kan fås genom att skapa och följa upp en systemanatomi?

Systemanatomin visade sig vara ett värdefullt verktyg i projektets inledande fas då den gav gruppen en gemensam översikt och förståelse för systemets huvudsakliga funktioner och beroenden. Genom att visualisera systemet kunde gruppen identifiera vilka delar av det som var centrala och vilka som kunde utvecklas senare, vilket underlättade prioritering och planering. Även om systemanatomin inte användes aktivt under hela projektets gång fungerade den som en strukturell grund och utgångspunkt vid uppstarten. Erfarenheten visar att en tidigt framtagen systemanatomi bidrar till gemensam förståelse för projektet och lägger grunden för en mer organiserad och fokuserad utvecklingsprocess. För framtida projekt rekommenderas därför att använda systemanatomin som ett kommunikations- och planeringsverktyg, särskilt i komplexa system med många beroenden.

7.4 Hur kan sprintplanering optimeras för att balansera utveckling, testning och dokumentation inom varje iteration?

För att uppnå en balanserad arbetsfördelning mellan utveckling, testning och dokumentation krävs en flexibel och iterativ sprintplanering där teamet kontinuerligt utvärderar och justerar sina arbetsmetoder. I EagleEye-projektet visade det sig effektivt att inleda med gemensam mobbprogrammering för att skapa en gemensam förståelse följt av övergång till par- och trioprogrammering för ökad produktivitet. Genom att successivt justera teamstorlekar och sprintlängder kunde arbetsbelastning och kommunikation förbättras. Verktyg som Jira och Kanban-tavlor underlättade uppföljning och prioritering av arbetsuppgifter, vilket skapade transparens och möjliggjorde snabb omfördelning av resurser. Sprintutvärderingar med fokus på förbättringsförslag bidrog också till att snabbt identifiera flaskhalsar och stärka balansen mellan olika arbetsmoment. En optimerad sprintplanering bör alltså ha regelbundna reflektioner, dynamisk teamstruktur och tydlig visualisering av arbetsflödet.



Litteratur

- [1] "RTSP Protocol - Real-time streaming: what is it and how does it work?" Hämtad: 2025-04-15. (), URL: <https://getstream.io/glossary/rtsp-protocol/>.
- [2] Amazon. "What is MQTT? - MQTT Protocol Explained." Hämtad: 2025-04-15. (), URL: <https://aws.amazon.com/what-is/mqtt/>.
- [3] A. Adegbenro. "What is WebRTC? (Explanation, use cases, and features)." Last updated: Aug 30, 2023. Hämtad: 2025-04-15. (2023), URL: <https://ably.com/blog/what-is-webrtc>.
- [4] A. Semochkin. "RTSPtoWebRTC." Hämtad: 2025-04-15. (2022), URL: <https://github.com/deepch/RTSPtoWebRTC>.
- [5] Meta. "React – A JavaScript library for building user interfaces." Hämtad: 2025-04-15. (), URL: <https://react.dev>.
- [6] G. Bierman, M. Abadi och M. Torgersen, "Understanding typescript," i *ECOOP 2014–Object-Oriented Programming: 28th European Conference, Uppsala, Sweden, July 28–August 1, 2014. Proceedings 28*, Springer, 2014, s. 257–281.
- [7] D. Muyltermans, "How does the virtual DOM compare to other DOM update mechanisms in JavaScript frameworks," *Daisy Muyltermans*, 2019.
- [8] Vite Contributors. "Getting Started | Vite Guide." [Online; hämtad: 22-maj-2025]. (2025), URL: <https://vite.dev/guide/>.
- [9] K. Schwaber och J. Sutherland, "The scrum guide," *Scrum Alliance*, årg. 21, nr 1, s. 1–38, 2011.
- [10] Atlassian. "What is Jira Software?" [Online; hämtad: 22-maj-2025]. (2025), URL: <https://www.atlassian.com/software/jira/guides/getting-started/introduction#what-is-jira-software>.
- [11] Figma Inc. "What is Figma?" [Online; hämtad: 22-maj-2025]. (2025), URL: <https://help.figma.com/hc/en-us/articles/14563969806359-What-is-Figma>.
- [12] S. Arachchi och I. Perera, "Continuous Integration and Continuous Delivery Pipeline Automation for Agile Software Project Management," i *2018 Moratuwa Engineering Research Conference (MERCon)*, 2018, s. 156–161. DOI: 10.1109/MERCon.2018.8421965.
- [13] The Go Authors. "Documentation." [Online; hämtad: 22-maj-2025]. (2025), URL: <https://go.dev/doc/>.

-
- [14] Python Software Foundation. "What is Python? Executive Summary." [Online; hämtad: 22-maj-2025]. (2025), URL: <https://www.python.org/doc/essays/blurb/>.
- [15] IBM. "Geographic coordinate system." [Online; hämtad: 22-maj-2025]. (2025), URL: <https://www.ibm.com/docs/en/informix-servers/12.10.0?topic=data-geographic-coordinate-system>.
- [16] T. Labs. "Tailwind CSS: A Utility-First CSS Framework." Hämtad: 2025-05-06. (2025), URL: <https://tailwindcss.com>.
- [17] M. Uppala. "Software Development: Comparative Study of Git Branching Models." Preprint publicerad på TechRxiv. (nov. 2024), URL: <https://www.techrxiv.org/doi/full/10.36227/techrxiv.172565536.64854061>.
- [18] Institutionen för datavetenskap, Linköpings universitet. "Architecture Notebook." Hämtad: 2025-05-07. (u.å), URL: https://www.ida.liu.se/~TDDC88/openup/practice.tech.evolutionary_arch.base/workproducts/architecture_notebook_9BB92433.html?nodeId=9351a72b.
- [19] D. Cerny och D. Dahlström, *Ett visualiseringsverktyg för att koordinera agil utveckling av en komplex produkt: En studie om framtagande och vidareutveckling av en systemanatomiför ökad gemensam förståelse i den agila utvecklingen på Volvo Cars*, 2019.
- [20] L. Taxén, "Att styra komplexa projekt med enkla bilder," *Verkstadsforum*, årg. 7, s. 23–, 2006.
- [21] L. Taxén, *The System Anatomy: Enabling Agile Project Management*. Lund: Studentlitteratur AB, 2011.
- [22] J. Dalton, "Kanban Board," i *Great Big Agile: An OS for Agile Leaders*. Berkeley, CA: Apress, 2019, s. 187–188, ISBN: 978-1-4842-4206-3. DOI: 10.1007/978-1-4842-4206-3_36. URL: https://doi.org/10.1007/978-1-4842-4206-3_36.
- [23] N. Damij och T. Damij, "An Approach to Optimizing Kanban Board Workflow and Shortening the Project Management Plan," *IEEE Transactions on Engineering Management*, årg. 71, s. 13 266–13 273, 2024. DOI: 10.1109/TEM.2021.3120984.
- [24] T. Sedano, P. Ralph och C. Péraire, "The Product Backlog," i *2019 IEEE/ACM 41st International Conference on Software Engineering (ICSE)*, 2019, s. 200–211. DOI: 10.1109/ICSE.2019.00036.
- [25] C. Becker, S. Betz, R. Chitchyan, L. Duboc, S. M. Easterbrook, B. Penzenstadler, N. Seyff och C. C. Venters, "Requirements: The Key to Sustainability," *IEEE Software*, årg. 33, nr 1, s. 56–65, 2016. DOI: 10.1109/MS.2015.158.
- [26] A. Communications. "Using Outdoor People Counting to Support Okazaki's Walkable City Center Concept." Hämtad: 2025-05-06. (2025), URL: <https://www.axis.com/customer-story/people-counting-walkable-city>.
- [27] E. Collini, L. A. I. Palesi, P. Nesi, G. Pantaleo och W. Zhao, "Flexible thermal camera solution for Smart city people detection and counting," *Multimedia Tools and Applications*, årg. 83, nr 7, s. 20 457–20 485, 2024.
- [28] Axis Communications. "media-stream-library-js." Hämtad: 2025-05-02. (2024), URL: <http://github.com/AxisCommunications/media-stream-library-js>.